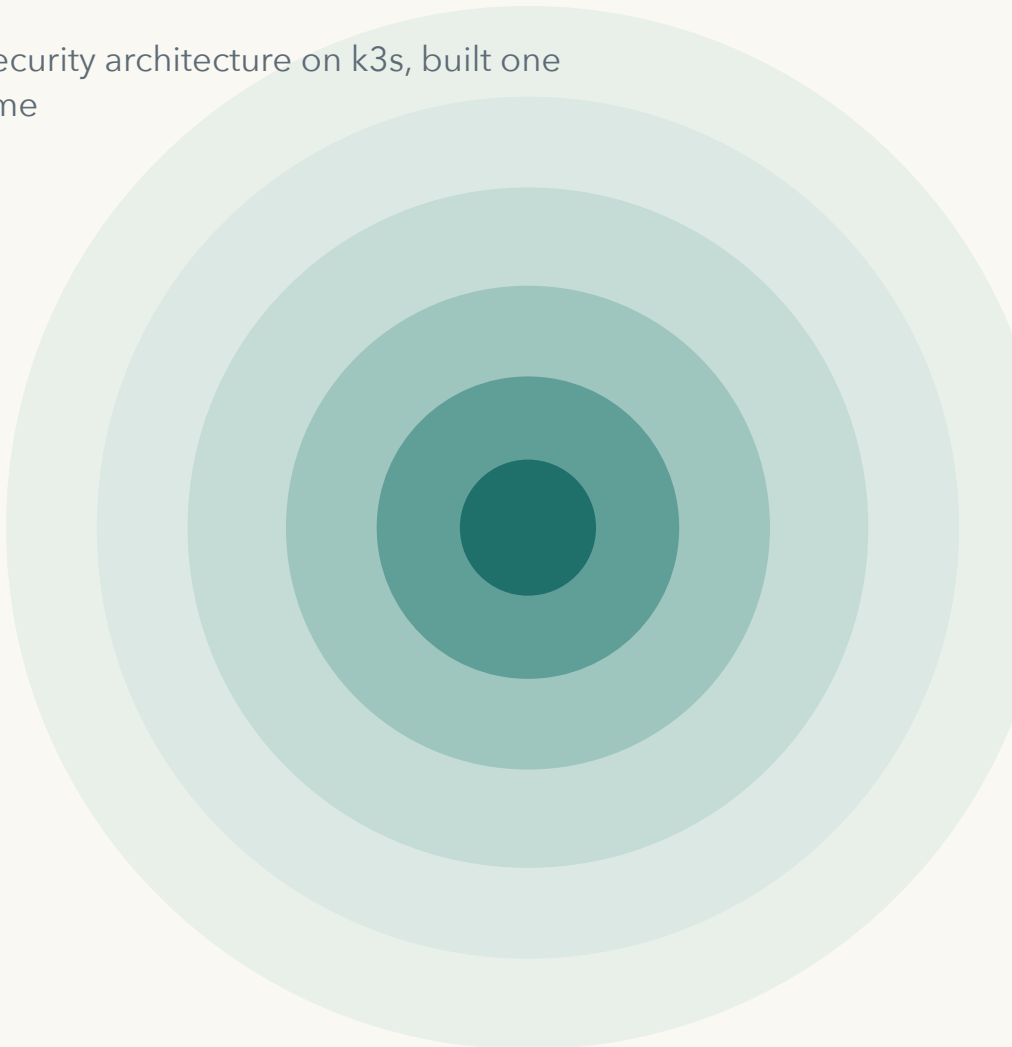
HANDS-ON GUIDE

Securing Kubernetes

An end-to-end security architecture on k3s, built one command at a time



Thomas Zachmann

As of November 6, 2026

Contents

How to Use This Book 6
Where You Run a Command 6
The Boxes 6
What Was Tested and What Was Not 7
What You Need 7
1 The Architecture at a Glance 8
1.1 The Core Principle: Defense in Layers 8
1.2 The Four Levels of Your Architecture 8
1.3 How This Book Is Structured 9
1.4 The Resource Plan 10
2 The Foundation: k3s and Cilium 12
2.1 What You Need 12
2.2 Preparing the VM 12
2.3 Installing k3s 13
2.4 Merging the Kubeconfig into Your Existing One 15
2.5 Using the New Context 16
2.6 Installing Cilium 17
2.7 Trying Out Hubble 18
3 RBAC: Who May Do What? 20
3.1 The Model 20
3.2 Creating the Lab Namespace 20
3.3 The Rights of a ServiceAccount 20
3.4 Not Mounting the Token in the First Place 21
3.5 A Real Person: alice 22
3.6 Recognizing Dangerous Rights 24
4 Pod Security Admission 26
4.1 The Bouncer 26
4.2 Who Needs Which Level? 26
4.3 Test the Effect Before It Hits 27
4.4 Set the Exemptions 27
4.5 Set the Cluster Default 28
4.6 Test the Default 29

4.7 Set `demo` to `restricted` 30
5 Kyverno: Your Own Rules 32
5.1 Why Kyverno When You Have PSA? 32
5.2 Installing 32
5.3 Writing the Policies 33
5.4 Testing Locally, Without a Cluster 34
5.5 Starting in Audit Mode 36
5.6 Enforcing 36
5.7 When Kyverno Fails 37
6 Default Deny with Cilium 40
6.1 Why Default Deny 40
6.2 The Lab Application 40
6.3 Step 1: Block Everything Except DNS 42
6.4 Step 2: Allow Specific Traffic 43
6.5 Step 3: Rules at the HTTP Level 44
6.6 Step 4: Allow Egress by Hostname 45
6.7 What Still Gets Through 46
6.8 How to Apply This to Real Namespaces 46
7 Falco: Detecting Suspicious Activity 48
7.1 The Alarm System in the Kernel 48
7.2 Installing 48
7.3 Simulating an Attack 49
7.4 Silencing False Positives 51
8 Trivy Operator: Scanning Regularly 54
8.1 Finding Known Weaknesses 54
8.2 Installing 54
8.3 Letting It Find an Old Image 55
8.4 Finding Misconfigurations 56
8.5 The Rule That Ties It All Together 57
9 Audit Logs: Who Did What? 58
9.1 The API Server's Security Camera 58
9.2 The Four Log Levels 58
9.3 Writing the Audit Policy 58
9.4 Enabling It on the Node 59

9.5 Analyzing the Log 60
9.6 Useful Queries 61
10 A Central View: Grafana and Loki 64
10.1 What You Build 64
10.2 Starting SeaweedFS 64
10.3 A Dedicated S3 Account for Loki 65
10.4 Installing Prometheus and Grafana 66
10.5 Installing Loki 68
10.6 Alloy Collects the Logs 70
10.7 Connecting Hubble and Trivy 72
10.8 Opening Grafana 73
10.9 Querying Audit Logs 74
10.10 Querying Falco Alerts 74
10.11 Querying Trivy Results 74
11 Tempo: Following a Request 76
11.1 Three Kinds of Signals 76
11.2 Bucket and S3 Account 76
11.3 Installing Tempo 77
11.4 Alloy Accepts Traces 78
11.5 The First Trace 80
11.6 The API Server Delivers Traces 80
11.7 Jumping from Traces to Logs 82
12 Bringing It All Together 84
12.1 The Security Dashboard 84
12.2 Alerts 85
12.3 The Attack Drill 86
12.4 The Investigation 88
12.5 What the Drill Showed 88
13 Operations and Troubleshooting 90
13.1 The Routine After Every Restart 90
13.2 What You Need to Back Up 90
13.3 The Troubleshooting Routine 91
13.4 The Emergency Brakes 91
13.5 Updates 92

13.6 Keeping an Eye on Resources 92
Optional Part 94
14 Optional: cert-manager 96
14.1 Why Automation 96
14.2 Installing 96
14.3 Building Your Own CA 97
14.4 The First Certificate 98
14.5 Trying Out Renewal 99
14.6 Making Grafana Reachable over HTTPS 99
15 Optional: OpenBao as a Vault 102
15.1 Why a Vault 102
15.2 Preparing the TLS Certificate 102
15.3 Installing 103
15.4 Initializing and Unsealing 105
15.5 First Steps in the Vault 105
15.6 Granting Permissions: the Policy 106
15.7 Experiencing the Seal 107
16 Optional: External Secrets 110
16.1 How the Pieces Fit Together 110
16.2 Installing 110
16.3 The Account for Access 111
16.4 OpenBao Trusts Kubernetes 111
16.5 Describing the Connection 112
16.6 The First External Secret 113
16.7 Changing and Following Along 113
16.8 Who May Do What Now? 114
16.9 Moving Loki's Credentials 115
Appendix 118
Versions 118
Your Repo at the End 118
Cheat Sheet 119
LogQL Cheat Sheet 119
Official Documentation 119

How to Use This Book

Installing individual security tools is quick. The hard part is something else: understanding which tool closes which gap, where they overlap, and what happens when one of them fails. That is exactly what you learn here, on your own cluster `k3s2`.

You build the architecture layer by layer. Every chapter follows the same flow: first you learn which problem is being solved. Then you type command after command, check the result, and try something yourself. There are no long scripts.

Where You Run a Command

Every command carries a colored label. It tells you where you are right now:

Label	Meaning
MAC	Your terminal on the laptop, in the root directory of this repo.
NODE	An SSH session on the VM `k3s2` (172.16.0.136).
POD	A shell in a container that you opened beforehand with `kubectl exec`.

Replace `<user>` with your SSH login each time. All paths like `security/psa.yaml` are relative to the repo.

The Boxes

WHY IT MATTERS

This is the background that helps you with the next problem.

PITFALL

This is what typically goes wrong and how you recognize it.

WHAT YOU SHOULD SEE

This is the expected result. If your output differs, do not move on. Clear up the difference first.

TRY IT YOURSELF

This is your turn. Think it through yourself first before you look things up.

What Was Tested and What Was Not

A book full of commands is only as good as its testing. So here is an honest account of how far that testing went.

Tested on November 6, 2026:

- All chart versions come from the official Helm repositories.
- All Helm values in this book were rendered with `helm template` against exactly these chart versions.
- The three Kyverno policies from Chapter 5 ran with the Kyverno CLI 1.19.1 against good and bad example pods.
- `ValidatingPolicy`, `ClusterSecretStore` and `ExternalSecret` were checked against the CRD schemas of the charts.
- The image tags of the lab workloads were looked up in the registry.

Not tested:

- The book was **not** run through end to end on a real cluster. The test cluster used while writing was still running Flannel and Kubernetes 1.37.
- The Alloy configuration, the Falco rules, the `CiliumNetworkPolicy` manifests, the TraceQL queries and the `bao` commands follow the official documentation, but did not run against the real programs.

So expect something to differ at some point. That is no reason to worry. It is practice: troubleshooting is part of learning, and Chapter 13 helps you with it.

What You Need

- A fresh Ubuntu 24.04 VM with 4 CPUs, 8 GB RAM and at least 40 GB of disk. Nothing besides Ubuntu is installed.
- On the laptop: `kubectl`, `helm`, `jq`, `openssl`, `curl` and the Kyverno CLI.
- This repo with `k3s/config.yaml` and `cilium/values.yaml`.
- About two weekends of time, or one chapter per evening.

A WORD ON THE ORDER

The book follows your plan: first RBAC and PSA, then Kyverno and network policies, then Falco, then Trivy, and finally audit logs with Grafana, Loki and Tempo. Chapter 1 explains why this works well even without early monitoring. cert-manager, OpenBao and External Secrets come at the end as an optional part.

CHAPTER 1

The Architecture at a Glance

You understand which layer stops which attack and why the build follows exactly this order.

Time: 20 minutes of reading

1.1 The Core Principle: Defense in Layers

No single measure is perfect. An attacker might find a hole in your web application, and suddenly there is code running in your cluster. Then the question is no longer whether something happened, but how far the attacker gets and whether you notice.

That is why you build several independent layers. Each one assumes that the one before it has failed. The journey of an attack shows how they work together:

Phase	What the attacker tries	Who steps in
Prepare	A vulnerable image runs in the cluster	Trivy, Kyverno
Break in	They exploit the hole and land in the pod	Falco reports it
Spread	They call other services or the internet	Cilium blocks
Escalate privileges	They use the service account token	RBAC limits
Break out	They start a privileged pod	PSA, Kyverno
Grab the loot	They search for passwords	RBAC, audit log
Cover tracks	They delete local logs	Loki keeps them elsewhere

1.2 The Four Levels of Your Architecture

It helps to order the tools by the point in time at which they act:

Level	Question	Tools
Before start	Is this allowed into the cluster at all?	RBAC, PSA, Kyverno
At runtime	What may it do while it runs?	Cilium, Falco
Continuously	Where are known weaknesses?	Trivy
Evidence	What actually happened?	Audit logs, Grafana, Loki, Tempo

This also shows you where tools overlap and where they do not:

- **PSA and Kyverno** both check at creation time. PSA knows only three fixed levels, Kyverno understands any custom rules you write. PSA is the simple foundation, Kyverno the addition.
- **Falco and Cilium** both act at runtime. Falco observes and reports, Cilium blocks. So Falco cannot prevent anything, but it sees things that never show up on the network.
- **Trivy and Kyverno** complement each other. Trivy finds weaknesses, Kyverno can prevent certain images from starting at all.
- **Loki and Tempo** store different things: Loki the log lines, Tempo the path of a single request through the system.

1.3 How This Book Is Structured

You work from the inside out: first the permissions, then the rules, then the network, then detection, and at the end you bring everything together in one place.

Ch.	Building block	Role
2	k3s and Cilium on the Ubuntu VM, kubeconfig	Foundation
3	RBAC	Access
4	Pod Security Admission	Admission
5	Kyverno	Admission
6	Cilium default deny	Runtime
7	Falco	Runtime
8	Trivy Operator	Continuous
9	Kubernetes audit logs	Evidence
10	Grafana, Prometheus, Loki	Central view
11	Tempo	Central view
12	Dashboard and attack exercise	Bringing it together
13	Operations and troubleshooting	Ongoing operations

Chapters 14 to 16 are an optional part on TLS and secrets with cert-manager, OpenBao and External Secrets. You do not need them for the main part.

Does This Work Without Monitoring at the Start?

Yes. When you lock things down, you need to see what is being blocked right now, and you can do that from the start:

Chapter	Your view while Grafana is still missing
RBAC	`kubectl auth can-i`
PSA	Warnings from `kubectl` , `--dry-run=server`
Kyverno	`kubectl get polr -A`
Cilium	`hubble observe --verdict DROPPED`
Falco	`kubectl logs`
Trivy	`kubectl get vuln -A`
Audit logs	`jq` directly on the node

There is one thing you have to plan ahead for: monitoring services need host access, so the `monitoring` namespace already gets its PSA exemption in Chapter 4, long before anything runs there.

The Rule in Every Chapter

Regardless of the tool, the same sequence always applies:

1. **Report.** Rules run in audit mode and block nothing.
2. **Evaluate.** You read the reports and adjust the rules.
3. **Enforce.** Only now is anything actually blocked.

1.4 The Resource Plan

Your VM has 8 GB of RAM. The table shows rough estimates of memory usage, not measurements. Compare them with `kubectl top pods -A` once everything is running.

Component	RAM (estimated)
k3s itself, CoreDNS, Traefik, Metrics Server	1.0 GB
Cilium, Hubble	0.5 GB
Kyverno	0.4 GB
Falco	0.3 GB
Trivy Operator (more for short periods during scans)	0.3 GB
Prometheus, Grafana, Alertmanager, exporters	1.5 GB
SeaweedFS, Loki, Alloy	0.8 GB
Tempo	0.3 GB
Total main part	**about 5.4 GB**

That leaves room for your lab application, but not much. The optional part needs about 0.3 GB more. You let Trivy run only one scan at a time; this is already configured in Chapter 8.

TRY IT YOURSELF

Take your own scenario: an attacker has a remote shell in the `frontend` pod. Write down three things they would try next, and note which layer should react in each case. In Chapter 12 you play through exactly this and compare.

WHAT YOU LEARNED

- Every layer assumes that the previous one has failed.
- Tools can be ordered by when they act: before start, at runtime, continuously and after the fact.
- Visibility does not start with Grafana: `kubectl` and Hubble are enough at the beginning.
- Report first, then evaluate, and enforce last.

CHAPTER 2

The Foundation: k3s and Cilium

An empty Ubuntu VM becomes a cluster with Cilium, and your laptop controls it through its own context.

Time: 60 minutes **Requires:** An Ubuntu VM with SSH access

2.1 What You Need

This chapter starts from zero. Nothing runs on the VM except Ubuntu.

Item	This book assumes
Operating system	Ubuntu 24.04
Size	4 CPUs, 8 GB RAM, at least 40 GB of disk
Name and address	`k3s2` with fixed IP `172.16.0.136`
Laptop	`kubectl`, `helm`, `jq`, `openssl`, `curl`, and this repo

The IP address is hard-coded in `cilium/values.yaml`. If your VM has a different one, change it there and replace it in all commands in this book. It must not change later, so give the VM a fixed IP or a DHCP reservation.

IS K3S ALREADY RUNNING ON THE VM?

Then remove it first with `sudo /usr/local/bin/k3s-uninstall.sh`. This deletes the cluster and its data irreversibly. With k3s, the network plugin is best chosen at installation time; rebuilding a running cluster is not worth it.

2.2 Preparing the VM

MAC Step 1

```
scp k3s/config.yaml <user>@172.16.0.136:/tmp/config.yaml
```

Copies your k3s configuration to the VM. It contains the flags that turn off Flannel and kube-proxy. We will look at it in a moment.

MAC Step 2

```
ssh <user>@172.16.0.136
```

Opens the session on the VM. All following steps with the **Node** label run here.

NODE Step 3

```
sudo hostnamectl set-hostname k3s2
```

The hostname becomes the node name in the cluster. So set it before the installation.

NODE Step 4

```
sudo apt update && sudo apt -y upgrade
```

Brings the system up to date. This is the very first security measure.

NODE Step 5

```
sudo apt install -y curl jq
```

`curl` fetches the installation script in a moment, and you need `jq` later for the audit logs.

NODE Step 6

```
uname -r
```

Shows the kernel version. Cilium requires at least 5.10; Ubuntu 24.04 ships 6.8.

NODE Step 7

```
sudo ufw status
```

Ubuntu's firewall is usually inactive. If it says `inactive`, just continue.

If the firewall is active, the API server, pods and services must get through:

NODE Step 8

```
sudo ufw allow 6443/tcp
sudo ufw allow from 10.42.0.0/16 to any
sudo ufw allow from 10.43.0.0/16 to any
```

These are the rules from the k3s docs. `10.42.0.0/16` is the pod network, `10.43.0.0/16` the service network. As an alternative, the k3s docs recommend turning `ufw` off completely. Keep port 22 open, or you lock yourself out.

2.3 Installing k3s

NODE Step 9

```
sudo install -D -m 600 -o root -g root \
  /tmp/config.yaml /etc/rancher/k3s/config.yaml
```

`install` copies the file and sets permissions in one step. `-D` creates missing directories, `-m 600` makes the file readable by root only. It must be in place **before** the installation, or k3s starts with Flannel.

NODE Step 10

```
rm /tmp/config.yaml
sudo grep -v '^#' /etc/rancher/k3s/config.yaml | grep .
```

Removes the copy and shows the active lines. Each of them has a purpose:

Setting	Purpose
`flannel-backend: "none"`	No default CNI, Cilium takes over
`disable-network-policy: true`	No second policy controller next to Cilium
`disable-kube-proxy: true`	Cilium also replaces kube-proxy
`cluster-cidr`, `service-cidr`	The networks, must match Cilium
`cluster-init: true`	etcd instead of SQLite, allows snapshots
`secrets-encryption: true`	Secrets are stored encrypted in the datastore

NODE Step 11

```
curl -sfL https://get.k3s.io | INSTALL_K3S_CHANNEL=v1.36 sh -
```

Installs k3s from the `v1.36` channel. This matters: the `stable` channel ships 1.37, but Cilium 1.20 is only tested up to 1.36.

NODE Step 12

```
sudo k3s kubectl get nodes
```

Queries the cluster directly on the VM.

WHAT YOU SHOULD SEE

The node `k3s2` shows `NotReady`, version `v1.36.x+k3s1`. That is correct, because there is no CNI yet. If k3s does not start, `sudo journalctl -u k3s -n 50` shows the reason.

NODE Step 13

```
sudo k3s secrets-encrypt status
```

Confirms that secrets are encrypted in the datastore.

NODE Step 14

```
sudo install -m 600 -o "$USER" \
  /etc/rancher/k3s/k3s.yaml /tmp/k3s2.yaml
```

The kubeconfig belongs to root. This creates a copy that your user may fetch with `scp`. Only you can read it, because it contains full cluster rights.

NODE Step 15

```
exit
```

Back to the laptop.

2.4 Merging the Kubeconfig into Your Existing One

`~/.kube/config` already contains other contexts. So you must not overwrite the file. Instead, you add the new cluster.

MAC Step 16

```
cp -p ~/.kube/config ~/.kube/config.bak-k3s2
```

Backup. If something goes wrong, you copy this file back.

MAC Step 17

```
scp <user>@172.16.0.136:/tmp/k3s2.yaml ~/.kube/k3s2.raw
ssh <user>@172.16.0.136 'rm /tmp/k3s2.yaml'
```

Fetches the file to the laptop and deletes the admin copy on the VM.

MAC Step 18

```
sed -e 's/127\.0\.0\.1/172.16.0.136/' \
    -e 's/: default$/: k3s2/' \
    ~/.kube/k3s2.raw > ~/.kube/k3s2.yaml
```

The file points to `127.0.0.1` and calls everything `default`. `sed` replaces the address and renames cluster, user and context to `k3s2`. That way nothing collides with your other contexts. The `$` only matches line endings, so the certificate data stays untouched.

MAC Step 19

```
chmod 600 ~/.kube/k3s2.yaml
```

Only you may read this file.

MAC Step 20

```
kubectl config get-contexts
```

Shows your contexts. If `k3s2` is already there from an earlier installation, you must remove the old entry:

MAC Step 21

```
kubectl config delete-context k3s2
kubectl config delete-cluster k3s2
kubectl config delete-user k3s2
```

Only needed if `k3s2` already exists. Old entries would win during the merge, and `kubectl` would then report `x509` errors.

MAC Step 22

```
KUBECONFIG=~/.kube/config:~/.kube/k3s2.yaml \
  kubectl config view --flatten > ~/.kube/config.neu
```

`KUBECONFIG` with two files makes `kubectl` merge both. `--flatten` writes the certificates directly into the output, so the file works on its own.

MAC Step 23

```
chmod 600 ~/.kube/config.neu
mv ~/.kube/config.neu ~/.kube/config
```

Puts the new file in place with the right permissions.

MAC Step 24

```
rm ~/.kube/k3s2.raw ~/.kube/k3s2.yaml
```

Cleans up the intermediate files with the admin credentials.

2.5 Using the New Context

MAC Step 25

```
kubectl config use-context k3s2
```

From now on, all `kubectl` and `helm` commands go to this cluster.

MAC Step 26

```
kubectl config get-contexts
kubectl get nodes
```

The asterisk marks the active context. Your other contexts are still there.

WHAT YOU SHOULD SEE

The asterisk is next to `k3s2`, and from the laptop you see the same `NotReady` node. From now on, always check the context before you delete anything.

2.6 Installing Cilium

Your `cilium/values.yaml` already contains everything needed. The two most important lines are `k8sServiceHost` and `k8sServicePort`.

WHY IT MATTERS

Without kube-proxy, at first there is nobody to translate the service IP `10.43.0.1`, since Cilium does that itself. But the Cilium agent has to reach the API server in order to start. The direct address `172.16.0.136:6443` solves this chicken-and-egg problem.

MAC — Step 27

```
helm upgrade --install cilium \
  oci://quay.io/cilium/charts/cilium \
  --version 1.20.4 -n kube-system \
  -f cilium/values.yaml --wait
```

`upgrade --install` works the first time and on every repeat. `--wait` only returns once all pods are ready.

MAC — Step 28

```
kubectl get nodes
```

Now the node should show `Ready`.

MAC — Step 29

```
kubectl -n kube-system exec ds/cilium -c cilium-agent -- \
  cilium-dbg status --brief
```

Asks the agent directly. `ds/cilium` picks a pod of the DaemonSet, `-c` the container.

MAC — Step 30

```
kubectl -n kube-system exec ds/cilium -c cilium-agent -- \
  cilium-dbg status | grep KubeProxyReplacement
```

Confirms that Cilium replaces kube-proxy. The expected value is `True`.

MAC — Step 31

```
kubectl run dns-test --rm -it --restart=Never \
  --image=busybox:1.36 -- \
  nslookup kubernetes.default.svc.cluster.local
```

Starts a throwaway pod and tests the pod network, DNS and service routing in one step. `--rm` removes it afterwards.

WHAT YOU SHOULD SEE

`nslookup` answers with the address `10.43.0.1`. This proves that CoreDNS is reachable and the service IP works without kube-proxy.

2.7 Trying Out Hubble

Hubble is Cilium's observability tool. It shows every network connection in the cluster along with the decision whether it was allowed. Until monitoring arrives in Chapter 10, it is your most important pair of eyes.

MAC Step 32

```
kubectl -n kube-system exec ds/cilium -c cilium-agent -- \
  hubble observe --last 10
```

Shows the last ten network flows on the node. The Hubble client is part of the agent image, so you do not need to install anything.

MAC Step 33

```
kubectl -n kube-system port-forward svc/hubble-ui 12000:80
```

Makes the graphical interface reachable at http://localhost:12000. Stop it with `Ctrl-C`.

TRY IT YOURSELF

In a second terminal, open `hubble observe --follow` (with the `exec` prefix from above) and start the `dns-test` again. Can you find the DNS request to `kube-dns` in the stream of flows?

WHAT YOU LEARNED

- You choose the CNI at installation time, not afterwards.
- The `config.yaml` must be on the VM before the installation.
- Without kube-proxy, Cilium must know the API server directly.
- `KUBECONFIG` with `--flatten` merges kubeconfigs without endangering other contexts.
- With Hubble, you now see every connection in the cluster.

CHAPTER 3

RBAC: Who May Do What?

You grant minimal rights to people and to workloads and check them with `kubectl auth can-i`.

Time: 60 minutes **Requires:** Chapter 2

3.1 The Model

RBAC (Role-Based Access Control) consists of four objects:

Object	Meaning
`Role`	Rights within a namespace
`ClusterRole`	Rights cluster-wide or as a reusable template
`RoleBinding`	Grants a role in a namespace
`ClusterRoleBinding`	Grants a role in the whole cluster

The principle is: everything that is not explicitly allowed is forbidden. Roles can only add rights, never take them away.

Three kinds of subjects receive rights: **User**, **Group** and **ServiceAccount**. Kubernetes has no user database. Users and groups come from the certificate or token someone logs in with. ServiceAccounts, on the other hand, are real objects in the cluster.

3.2 Creating the Lab Namespace

MAC Step 1

```
kubectl create namespace demo
```

You practice in this namespace for the rest of the book.

MAC Step 2

```
kubectl label namespace demo security.k3s2/tier=app
```

Kyverno uses this label in Chapter 5 to recognize application namespaces.

3.3 The Rights of a ServiceAccount

Every pod runs under a ServiceAccount. If none is specified, that is `default`.

MAC Step 3

```
kubectl auth can-i --list \
  --as=system:serviceaccount:demo:default -n demo
```

`--as` lets you ask as this account without logging in. This is called impersonation. You see almost nothing but rights for self-discovery.

MAC Step 4

```
kubectl create serviceaccount app -n demo
```

A dedicated account for your application. That way it shares its rights with nobody.

MAC Step 5

```
kubectl create role pod-leser -n demo \
  --verb=get,list,watch --resource=pods
```

Allows only reading pods in `demo`.

MAC Step 6

```
kubectl create rolebinding app-liest-pods -n demo \
  --role=pod-leser --serviceaccount=demo:app
```

Grants the role to the `app` account. Only the binding makes the role effective.

MAC Step 7

```
kubectl auth can-i list pods -n demo \
  --as=system:serviceaccount:demo:app
```

Expected: `yes`.

MAC Step 8

```
kubectl auth can-i delete pods -n demo \
  --as=system:serviceaccount:demo:app
```

Expected: `no`. Reading was allowed, deleting is not.

MAC Step 9

```
kubectl auth can-i list pods -n kube-system \
  --as=system:serviceaccount:demo:app
```

Expected: `no`. The role only applies in `demo`.

3.4 Not Mounting the Token in the First Place

Kubernetes automatically puts the ServiceAccount token into every pod. But most applications never talk to the API server. For them, the token is just a gift to the attacker.

FILE `security/rbac/sa-ohne-token.yaml`

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: frontend
  namespace: demo
automountServiceAccountToken: false
```

Pods using this account get no token. You can override this per pod if one really needs it.

MAC Step 10

```
kubectl apply -f security/rbac/sa-ohne-token.yaml
```

The `frontend` uses this account in Chapter 6.

3.5 A Real Person: alice

Now you give a person access, through a client certificate. `CN` becomes the user name, `O` the group.

MAC Step 11

```
mkdir -p ~/k3s2-alice && cd ~/k3s2-alice
```

A separate folder outside the repo, because a private key will be stored here in a moment.

MAC Step 12

```
openssl genrsa -out alice.key 2048
```

Creates alice's private key. It never leaves her machine.

MAC Step 13

```
openssl req -new -key alice.key -out alice.csr \
  -subj "/CN=alice/O=plattform"
```

Creates a certificate signing request for user `alice` in group `plattform`.

Create the request as a Kubernetes object:

FILE `~/k3s2-alice/csr.yaml`

```
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata:
  name: alice
spec:
  request: HIER_BASE64
  signerName: kubernetes.io/kube-apiserver-client
  expirationSeconds: 86400
  usages:
    - client auth
```

`expirationSeconds: 86400` limits the certificate to one day. This matters because Kubernetes **cannot revoke** client certificates.

MAC — Step 14

```
base64 < alice.csr | tr -d '\n'
```

Prints the request as a single Base64 line. Put it into `csr.yaml` in place of `HIER_BASE64`.

MAC — Step 15

```
kubectl apply -f csr.yaml
kubectl certificate approve alice
```

Submits the request and approves it. Now the cluster signs it.

MAC — Step 16

```
kubectl get csr alice \
  -o jsonpath='{.status.certificate}' | base64 -d > alice.crt
```

Fetches the finished certificate.

MAC — Step 17

```
kubectl config view --raw --minify \
  -o jsonpath='{.clusters[0].cluster.certificate-authority-data}' \
  | base64 -d > k3s2-ca.crt
```

Saves the cluster's CA certificate. alice uses it to verify that she is really talking to your API server. It is public, not secret.

MAC — Step 18

```
kubectl config set-cluster k3s2 --kubeconfig=alice.kubeconfig \
  --server=https://172.16.0.136:6443 \
  --certificate-authority=k3s2-ca.crt --embed-certs
```

Adds the cluster to a new kubeconfig. Your own stays untouched.

MAC — Step 19

```
kubectl config set-credentials alice \
  --kubeconfig=alice.kubeconfig \
  --client-certificate=alice.crt --client-key=alice.key \
  --embed-certs
```

Adds alice's identity.

MAC — Step 20

```
kubectl config set-context alice --kubeconfig=alice.kubeconfig \
  --cluster=k3s2 --user=alice --namespace=demo
kubectl config use-context alice --kubeconfig=alice.kubeconfig
```

Connects cluster and user into a context.

MAC Step 21

```
kubectl --kubeconfig=alice.kubeconfig get pods
```

Expected: `Forbidden`. That is correct, because alice is logged in but has no rights yet. Authentication (who are you?) and authorization (what may you do?) are two steps.

MAC Step 22

```
kubectl create rolebinding plattform-edit -n demo \
  --clusterrole=edit --group=plattform
```

Grants the built-in `edit` role to the **group**. That way every person in this group gets the rights, and you do not have to maintain bindings per person.

MAC Step 23

```
kubectl --kubeconfig=alice.kubeconfig get pods
kubectl --kubeconfig=alice.kubeconfig get nodes
```

Pods work now. Nodes are still forbidden, because they are cluster-wide, and the binding only applies in `demo`.

MAC Step 24

```
cd -
```

Back to the repo.

3.6 Recognizing Dangerous Rights

Some rights look harmless but amount to full access:

Right	Why it is dangerous
`get secrets`	Reads all tokens and passwords in the namespace
`create pods`	Starts pods with any ServiceAccount in the namespace
`pods/exec`	Opens a shell in other people's containers
`escalate`, `bind`	Grants rights you do not have yourself
`impersonate`	Acts as any other user

MAC Step 25

```
kubectl get clusterrolebindings -o json | jq -r '
  .items[]
  | select(.roleRef.name == "cluster-admin")
  | .metadata.name'
```

Lists all bindings with full admin rights. This is your most important checklist: whatever appears here controls everything.

TRY IT YOURSELF

Why is `create pods` so dangerous? Imagine there is a ServiceAccount with high privileges in the namespace. How would a user who may only create pods get hold of its rights? The section on the token in the pod helps you here.

WHAT YOU LEARNED

- Roles only add rights; everything else is forbidden.
- Authentication and authorization are separate steps.
- `kubectl auth can-i --as` checks rights without logging in.
- Bindings to groups are better than bindings to individuals.
- Client certificates cannot be revoked, so keep them short-lived.

CHAPTER 4

Pod Security Admission

You decide which pods are allowed to start at all: a cluster-wide default, plus stricter or looser rules per namespace.

Time: 45 minutes **Requires:** Chapter 3

4.1 The Bouncer

Pod Security Admission (PSA) checks every new pod before it is created. It is built into Kubernetes. You choose a level and a mode per namespace.

Level	Meaning
`privileged`	No restrictions. For system components.
`baseline`	Blocks known dangers such as `privileged`, `hostNetwork`, `hostPath`.
`restricted`	The strictest: no root, capabilities dropped, seccomp required.

Mode	Effect of a violation
`enforce`	The pod is rejected.
`audit`	The pod starts, and the violation goes to the audit log.
`warn`	The pod starts, and `kubectl` shows a warning.

WHY IT MATTERS

PSA only checks **at creation time**. Running pods stay untouched, even if you tighten the level. A problem only shows up at the next restart. So check beforehand who a change would affect.

4.2 Who Needs Which Level?

Falco has to watch the kernel and needs a privileged container with many `hostPath` mounts. `restricted` would reject it right away. The same goes for anything that inspects the node itself.

Namespace	Level	Reason
`kube-system`	exempt	Cilium, Traefik, CoreDNS need host access
`monitoring`	`privileged`	node-exporter uses `hostNetwork` and `hostPID`, Alloy uses `hostPath`

Namespace	Level	Reason
`falco`	`privileged`	eBPF driver, many `hostPath` mounts
`trivy-system`	`privileged`	The node collector checks host files
`demo`	`restricted`	Your applications
all others	`baseline`	Cluster-wide default

The default also applies to new namespaces. Anyone who needs an exemption has to ask for it explicitly. That is the principle of **secure by default**.

4.3 Test the Effect Before It Hits

MAC Step 1

```
kubectl label --dry-run=server --overwrite ns --all \
  pod-security.kubernetes.io/enforce=baseline
```

`--dry-run=server` lets the API server check the change without saving it. It warns you about every running pod that violates `baseline`.

WHAT YOU SHOULD SEE

You get warnings for `kube-system`, for example because of `hostPath` or `hostNetwork`. This namespace needs its exemption. No other host-level services are running yet; they come later.

4.4 Set the Exemptions

Set the exemptions **first**, including for services that only arrive later. Otherwise the new default rejects them as soon as they try to start.

MAC Step 2

```
kubectl create namespace monitoring
kubectl create namespace falco
kubectl create namespace trivy-system
```

Creates the three namespaces now, so they have their level before the first pods start in them.

MAC Step 3

```
kubectl label namespace monitoring \
  pod-security.kubernetes.io/enforce=privileged
```

Allows everything in `monitoring`. Labels on the namespace override the cluster-wide default.

MAC Step 4

```
kubectl label namespace monitoring \
  pod-security.kubernetes.io/warn=baseline \
  pod-security.kubernetes.io/audit=baseline
```

Blocks nothing, but makes violations visible. That keeps the exemption traceable.

MAC Step 5

```
kubectl label namespace falco trivy-system \
  pod-security.kubernetes.io/enforce=privileged
```

Gives both the level they need.

4.5 Set the Cluster Default

FILE security/psa.yaml

```
apiVersion: apiserver.config.k8s.io/v1
kind: AdmissionConfiguration
plugins:
  - name: PodSecurity
    configuration:
      apiVersion: pod-security.admission.config.k8s.io/v1
      kind: PodSecurityConfiguration
      defaults:
        enforce: baseline
        enforce-version: latest
        audit: restricted
        audit-version: latest
        warn: restricted
        warn-version: latest
      exemptions:
        usernames: []
        runtimeClasses: []
        namespaces:
          - kube-system
```

The default enforces `baseline` and reports everything that would not meet `restricted`. Only `kube-system` is fully exempt. Exemptions apply to whole namespaces, so keep the list short.

MAC Step 6

```
scp security/psa.yaml <user>@172.16.0.136:/tmp/psa.yaml
```

Copies the file to the node.

Now uncomment these two lines in `k3s/config.yaml`. The `audit-` lines stay commented out; they are for Chapter 9.

FILE k3s/config.yaml (excerpt)

```
kube-apiserver-arg:
  - "admission-control-config-file=/var/lib/rancher/k3s/server/psa.yaml"
```

MAC Step 7

```
scp k3s/config.yaml <user>@172.16.0.136:/tmp/config.yaml
ssh <user>@172.16.0.136
```

Transfers the configuration and opens the session.

NODE Step 8

```
sudo install -m 600 -o root -g root /tmp/psa.yaml \
  /var/lib/rancher/k3s/server/psa.yaml
```

The file has to be in place before the restart.

NODE Step 9

```
sudo install -m 600 -o root -g root /tmp/config.yaml \
  /etc/rancher/k3s/config.yaml
rm /tmp/psa.yaml /tmp/config.yaml
sudo systemctl restart k3s
```

Installs the configuration and restarts k3s.

NODE Step 10

```
exit
```

Back to the Mac.

4.6 Test the Default

MAC Step 11

```
kubectl create namespace psa-test
```

A fresh namespace without a label, so the cluster default applies here.

FILE `security/psa-boese.yaml`

```
apiVersion: v1
kind: Pod
metadata:
  name: boese
  namespace: psa-test
spec:
  hostNetwork: true
  containers:
    - name: c
      image: busybox:1.36
      command: [sleep, "3600"]
      securityContext:
        privileged: true
```

MAC Step 12

```
kubectl apply -f security/psa-boese.yaml
```

Expected: `Forbidden`. The message names both violations, `hostNetwork` and `privileged`.

MAC Step 13

```
kubectl delete namespace psa-test
```

Cleans up.

4.7 Set `demo` to `restricted`

MAC Step 14

```
kubectl label namespace demo \
  pod-security.kubernetes.io/enforce=restricted
```

From now on, `demo` only runs pods that follow every rule of the strictest level.

MAC Step 15

```
kubectl run test --image=nginx:1.29-alpine -n demo
```

Expected: `Forbidden`. Read the message carefully, because it tells you which fields are missing.

Field in the pod	Purpose
`allowPrivilegeEscalation: false`	Processes cannot gain extra privileges
`capabilities.drop: [ALL]`	Removes all special Linux privileges
`runAsNonRoot: true`	The container does not start as root
`seccompProfile.type: RuntimeDefault`	Blocks rare, dangerous system calls

Standard `nginx` starts as root and therefore does not run here. You need an image that works without root:

FILE `security/psa-gut.yaml`

```
apiVersion: v1
kind: Pod
metadata:
  name: gut
  namespace: demo
  labels:
    team: plattform
spec:
  securityContext:
    runAsNonRoot: true
    seccompProfile:
      type: RuntimeDefault
  containers:
    - name: web
      image: nginxinc/nginx-unprivileged:1.29-alpine
      securityContext:
        allowPrivilegeEscalation: false
        capabilities:
```

```
          drop: [ALL]
      resources:
        limits:
          memory: 64Mi
```

MAC Step 16

```
kubectl apply -f security/psa-gut.yaml
kubectl get pod gut -n demo
```

This pod meets all the rules and runs.

MAC Step 17

```
kubectl delete pod gut -n demo
```

Cleans up.

TRY IT YOURSELF

Start a standard `nginx` pod in a new namespace. It passes `baseline` but violates `restricted`. Which warning does `kubectl` show? The `audit` mode writes the same violation to the audit log, which you enable in Chapter 9.

WHAT YOU LEARNED

- PSA only checks at creation time; running pods stay untouched.
- `--dry-run=server` shows in advance who a tighter level would affect.
- Set the exemptions first, then tighten the default.
- The `baseline` default makes new namespaces safer automatically.
- `restricted` needs four settings and an image that runs without root.

CHAPTER 5

Kyverno: Your Own Rules

You write policies that PSA does not know about, test them locally and roll them out step by step.

Time: 60 minutes **Requires:** Chapter 4

5.1 Why Kyverno When You Have PSA?

PSA only knows three fixed levels and says nothing about **what** should apply in your cluster. Kyverno fills exactly that gap, for example:

- Images need a fixed tag, not `latest`.
- Every container needs a memory limit.
- Every pod carries the `team` label.

Kyverno is an admission webhook: the API server asks it for permission before saving. A rule is a `ValidatingPolicy` with a CEL expression. CEL is the expression language that Kubernetes itself uses as well.

PITFALL

Many guides still use `kind: ClusterPolicy`. That has been deprecated since Kyverno 1.19 and will be removed in 1.20. Use `ValidatingPolicy` with `apiVersion: policies.kyverno.io/v1`.

5.2 Installing

FILE `kyverno/values.yaml`

```yaml
admissionController:
  replicas: 1
backgroundController:
  replicas: 1
cleanupController:
  replicas: 1
reportsController:
  replicas: 1
```

One node, so one instance each. That saves RAM, but it also means: if the pod fails, Kyverno is briefly unreachable.

MAC Step 1

```
helm repo add kyverno https://kyverno.github.io/kyverno/
helm repo update
```

Adds the repository.

MAC Step 2

```
helm upgrade --install kyverno kyverno/kyverno \
  -n kyverno --create-namespace --version 3.9.2 \
  -f kyverno/values.yaml --wait --timeout 5m
```

Chart 3.9.2 contains Kyverno 1.19.2. The new namespace automatically gets the `baseline` level.

MAC Step 3

```
kubectl get pods -n kyverno
```

Four controllers should be running.

5.3 Writing the Policies

FILE `security/kyverno/policies.yaml`

```
apiVersion: policies.kyverno.io/v1
kind: ValidatingPolicy
metadata:
  name: disallow-latest-tag
spec:
  validationActions: [Audit]
  matchConstraints:
    namespaceSelector:
      matchExpressions:
        - key: kubernetes.io/metadata.name
          operator: NotIn
          values: [kube-system, kyverno]
    resourceRules:
      - apiGroups: ['']
        apiVersions: [v1]
        operations: [CREATE, UPDATE]
        resources: [pods]
  validations:
    - message: Images need a fixed tag, not latest.
      expression: >-
        object.spec.containers.all(c,
          c.image.contains(':') && !c.image.endsWith(':latest'))
---
apiVersion: policies.kyverno.io/v1
kind: ValidatingPolicy
metadata:
  name: require-memory-limit
spec:
  validationActions: [Audit]
  matchConstraints:
    namespaceSelector:
      matchLabels:
        security.k3s2/tier: app
    resourceRules:
```

```
      - apiGroups: ['']
        apiVersions: [v1]
        operations: [CREATE, UPDATE]
        resources: [pods]
  validations:
    - message: Every container needs a memory limit.
      expression: >-
        object.spec.containers.all(c,
          c.?resources.?limits.?memory.orValue('') != '')
---
apiVersion: policies.kyverno.io/v1
kind: ValidatingPolicy
metadata:
  name: require-team-label
spec:
  validationActions: [Audit]
  matchConstraints:
    namespaceSelector:
      matchLabels:
        security.k3s2/tier: app
    resourceRules:
      - apiGroups: ['']
        apiVersions: [v1]
        operations: [CREATE, UPDATE]
        resources: [pods]
  validations:
    - message: The team label is missing.
      expression: >-
        object.metadata.?labels.team.orValue('') != ''
```

Here is how to read the rule:

- `validationActions: [Audit]` only reports. `Deny` would block.
- `matchConstraints` defines who the rule applies to. The first rule applies almost everywhere, the others only in namespaces with the label `security.k3s2/tier=app`.
- `expression` is the CEL expression. It must evaluate to `true` for the pod to pass.
- `.all(c, ...)` checks every container, and `?.` safely accesses fields that may be missing. `orValue('')` then provides a fallback value.

PITFALL

The `disallow-latest-tag` rule checks for a colon. An image like `registry:5000/app` does have a colon, but no tag, and would slip through. That is fine for a lab; in production you would check for digests right away.

5.4 Testing Locally, Without a Cluster

The Kyverno CLI runs policies without a cluster. That way you find mistakes before they bother anyone.

FILE security/kyverno/test/gut.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: gut
  namespace: demo
  labels:
    team: plattform
spec:
  containers:
    - name: web
      image: nginxinc/nginx-unprivileged:1.29-alpine
      resources:
        limits:
          memory: 64Mi
```

FILE security/kyverno/test/schlecht.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: schlecht
  namespace: demo
spec:
  containers:
    - name: web
      image: nginx:latest
```

FILE security/kyverno/test/values.yaml

```yaml
apiVersion: cli.kyverno.io/v1alpha1
kind: Values
namespaceSelector:
  - name: demo
    labels:
      kubernetes.io/metadata.name: demo
      security.k3s2/tier: app
```

Offline, the CLI knows nothing about namespaces. This file tells it which labels `demo` has.

MAC

Step 4

```
kyverno apply security/kyverno/policies.yaml \
  --resource security/kyverno/test/gut.yaml \
  --values-file security/kyverno/test/values.yaml
```

Checks the good pod. You already installed the CLI (`brew install kyverno`).

WHAT YOU SHOULD SEE

The result is `pass: 3, fail: 0` . For the `schlecht.yaml` example it is `pass: 0, fail: 3` , each with your error message. That is exactly what came out when this book was tested.

MAC Step 5

```
kyverno apply security/kyverno/policies.yaml \
  --resource security/kyverno/test/schlecht.yaml \
  --values-file security/kyverno/test/values.yaml
```

Checks the bad pod. All three rules should fire.

5.5 Starting in Audit Mode

MAC Step 6

```
kubectl apply -f security/kyverno/policies.yaml
```

Loads the three rules. They do not block anything yet.

MAC Step 7

```
kubectl get vpol
```

`vpol` is the short form of `validatingpolicies`. All three should be ready.

MAC Step 8

```
kubectl get polr -A
```

`polr` stands for `policyreports`. Kyverno also checks existing pods in the background and writes the result here. The `PASS` and `FAIL` columns show the status per resource.

MAC Step 9

```
kubectl get polr -A -o json | jq -r '
  .items[].results[]?
  | select(.result == "fail")
  | "\(.policy): \(.message)"' | sort | uniq -c
```

Summarizes all violations. This is your to-do list before you enforce anything.

5.6 Enforcing

If the report looks good, you switch over. Nothing is running in `demo` right now, so you can start safely.

In `security/kyverno/policies.yaml`, change the line for `require-memory-limit` and `require-team-label` to `validationActions: [Deny]`. The `latest` tag rule stays on `Audit` for now, because it affects almost the entire cluster.

MAC Step 10

```
kubectl apply -f security/kyverno/policies.yaml
```

Turns on enforcement.

MAC Step 11

```
kubectl run verboten -n demo \
  --image=nginxinc/nginx-unprivileged:1.29-alpine
```

Expect a rejection, probably from PSA first, because the `restricted` fields are missing here too. This nicely shows that both layers check independently.

MAC Step 12

```
kubectl apply --dry-run=server -f security/kyverno/test/gut.yaml
```

Checks the pod from the test without creating it. PSA rejects it, because the file does not meet `restricted`. Kyverno alone would have let it through.

PITFALL

The rules apply to pods. A deployment with a violation is often accepted at first, and only its pods fail. The deployment then shows `0/1`, and the cause is in the events: `kubectl get events -n demo`. Whether Kyverno also rejects the deployment directly is controlled by `spec.autogen`; see the docs.

5.7 When Kyverno Fails

A webhook can get in the cluster's way: if it is set to reject requests and is unreachable, no pod starts anymore. So remember the emergency exit:

MAC Step 13

```
kubectl get validatingwebhookconfigurations
```

Lists all webhooks. In an emergency, you delete the affected entry with `kubectl delete`. Kyverno recreates it on its next start.

TRY IT YOURSELF

Write a fourth rule, `disallow-default-sa`: pods in app namespaces must not use the `default` ServiceAccount. Hint: the field is called `object.spec.serviceAccountName`, and it may also be missing entirely. Test it with the CLI first and extend your test pods.

WHAT YOU LEARNED

- PSA provides the foundation, Kyverno the rules for your cluster.
- `ValidatingPolicy` replaces the deprecated `ClusterPolicy`.
- With the CLI you test policies locally, without a cluster.
- Policy reports show you in advance what `Deny` would hit.
- Admission webhooks are critical for the cluster, so know your emergency exit.

CHAPTER 6

Default Deny with Cilium

Pods can only do what you explicitly allow, and Hubble shows you every blocked connection.

Time: 75 minutes **Requires:** Chapter 5

6.1 Why Default Deny

By default, in Kubernetes every pod can talk to every other pod and reach the internet. For an attacker, that is ideal: one pod is enough, and from there they can reach everything. With **default deny** you turn this around: nothing works except what you allow.

Cilium works with identities instead of IP addresses. A rule says "pods with label `app=frontend`", not "10.42.0.7". That fits Kubernetes, where IP addresses change all the time.

WHY IT MATTERS

As soon as a policy selects a pod, default deny automatically applies to that direction. An `ingress` rule therefore blocks all other incoming traffic. That is powerful, but it is also the most common cause of surprises.

6.2 The Lab Application

Your application has two parts: a `backend` (web server) and a `frontend` that calls it. Both meet `restricted` and your Kyverno rules.

FILE `security/demo/app.yaml`

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  namespace: demo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
        team: plattform
    spec:
```

```
      automountServiceAccountToken: false
      securityContext:
        runAsNonRoot: true
        seccompProfile:
          type: RuntimeDefault
      containers:
        - name: web
          image: nginxinc/nginx-unprivileged:1.29-alpine
          ports:
            - containerPort: 8080
          securityContext:
            allowPrivilegeEscalation: false
            capabilities:
              drop: [ALL]
          resources:
            limits:
              memory: 64Mi
---
apiVersion: v1
kind: Service
metadata:
  name: backend
  namespace: demo
spec:
  selector:
    app: backend
  ports:
    - port: 8080
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
  namespace: demo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
        team: plattform
    spec:
      serviceAccountName: frontend
      securityContext:
        runAsNonRoot: true
        runAsUser: 100
        seccompProfile:
          type: RuntimeDefault
      containers:
        - name: curl
          image: curlimages/curl:8.17.0
          command: [sleep, "86400"]
          securityContext:
            allowPrivilegeEscalation: false
            capabilities:
              drop: [ALL]
          resources:
            limits:
              memory: 32Mi
```

MAC Step 1

```
kubectl apply -f security/demo/app.yaml
kubectl get pods -n demo
```

Both pods should show `Running`. If not, `kubectl get events -n demo` shows the reason.

MAC Step 2

```
kubectl exec -n demo deploy/frontend -- \
  curl -s -m 3 -o /dev/null -w '%{http_code}\n' http://backend:8080/
```

Calls the backend. `-m 3` gives up after three seconds, `-w` prints only the status code. Expected: `200`.

MAC Step 3

```
kubectl exec -n demo deploy/frontend -- \
  curl -s -m 5 -o /dev/null -w '%{http_code}\n' https://example.com/
```

The internet is reachable too. Expected: `200`. There are no rules yet.

6.3 Step 1: Block Everything Except DNS

FILE `security/netpol/01-default-deny.yaml`

```
apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
metadata:
  name: default-deny
  namespace: demo
spec:
  description: Block everything, only DNS stays allowed
  endpointSelector: {}
  ingress:
    - {}
  egress:
    - toEndpoints:
        - matchLabels:
            io.kubernetes.pod.namespace: kube-system
            k8s-app: kube-dns
      toPorts:
        - ports:
            - port: "53"
              protocol: ANY
          rules:
            dns:
              - matchPattern: "*"
```

`endpointSelector: {}` selects all pods in the namespace. `ingress: [{}]` is an empty rule: it allows nothing, but it turns on default deny for incoming traffic. For `egress`, only DNS stays allowed; otherwise no pod could resolve names anymore. The `dns` rule lets Cilium read along with the queries, which you will need shortly for hostnames.

MAC Step 4

```
kubectl apply -f security/netpol/01-default-deny.yaml
```

From now on, `demo` is locked down.

MAC Step 5

```
kubectl exec -n demo deploy/frontend -- \
  curl -s -m 3 -o /dev/null -w '%{http_code}\n' http://backend:8080/
```

Expected: `000` and a timeout. The name resolves, but the connection is silently dropped.

MAC Step 6

```
kubectl -n kube-system exec ds/cilium -c cilium-agent -- \
  hubble observe --namespace demo --verdict DROPPED --last 10
```

Hubble shows you exactly what was blocked: source, destination, port and reason.

WHAT YOU SHOULD SEE

One line shows `demo/frontend-... -> demo/backend-...:8080`, along with `Policy denied` and `DROPPED`. This is your most important tool for network problems: instead of guessing, you just look.

6.4 Step 2: Allow Specific Traffic

A connection needs two rules: the frontend must be allowed to send, and the backend must be allowed to receive.

FILE `security/netpol/02-frontend-backend.yaml`

```yaml
apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
metadata:
  name: frontend-darf-senden
  namespace: demo
spec:
  endpointSelector:
    matchLabels:
      app: frontend
  egress:
    - toEndpoints:
        - matchLabels:
            app: backend
      toPorts:
        - ports:
            - port: "8080"
              protocol: TCP
---
apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
metadata:
  name: backend-darf-empfangen
  namespace: demo
spec:
  endpointSelector:
    matchLabels:
      app: backend
  ingress:
```

```
    - fromEndpoints:
        - matchLabels:
            app: frontend
      toPorts:
        - ports:
            - port: "8080"
              protocol: TCP
```

Policies add up. The block stays in place; these rules only add exceptions. Without a namespace, the labels only apply within the policy's own namespace.

MAC Step 7

```
kubectl apply -f security/netpol/02-frontend-backend.yaml
```

Opens exactly this one connection.

MAC Step 8

```
kubectl exec -n demo deploy/frontend -- \
  curl -s -m 3 -o /dev/null -w '%{http_code}\n' http://backend:8080/
```

Expected: `200`.

MAC Step 9

```
kubectl exec -n demo deploy/frontend -- \
  curl -s -m 5 -o /dev/null -w '%{http_code}\n' https://example.com/
```

Expected: `000`. The internet stays blocked.

6.5 Step 3: Rules at the HTTP Level

Cilium can also understand HTTP. That lets you allow specific paths, not just a port. To do this, change the `backend-darf-empfangen` rule:

FILE `security/netpol/03-http-regel.yaml`

```
apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
metadata:
  name: backend-darf-empfangen
  namespace: demo
spec:
  endpointSelector:
    matchLabels:
      app: backend
  ingress:
    - fromEndpoints:
        - matchLabels:
            app: frontend
      toPorts:
        - ports:
            - port: "8080"
              protocol: TCP
          rules:
```

```
            http:
              - method: GET
                path: "/"
```

MAC Step 10

```
kubectl apply -f security/netpol/03-http-regel.yaml
```

The name is the same, so this command replaces the rule.

MAC Step 11

```
kubectl exec -n demo deploy/frontend -- \
  curl -s -m 3 -o /dev/null -w '%{http_code}\n' http://backend:8080/
```

Expected: `200`.

MAC Step 12

```
kubectl exec -n demo deploy/frontend -- \
  curl -s -m 3 -o /dev/null -w '%{http_code}\n' http://backend:8080/admin
```

Expected: `403`. The difference from before matters: the connection is established, but Cilium's proxy rejects the request.

MAC Step 13

```
kubectl exec -n demo deploy/frontend -- \
  curl -s -m 3 -o /dev/null -w '%{http_code}\n' \
  -X DELETE http://backend:8080/
```

Expected: `403`. Only `GET` is allowed.

6.6 Step 4: Allow Egress by Hostname

Often an application needs to reach a specific service on the internet. IP addresses change, so you specify the name instead.

FILE `security/netpol/04-fqdn.yaml`

```
apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
metadata:
  name: frontend-darf-example
  namespace: demo
spec:
  endpointSelector:
    matchLabels:
      app: frontend
  egress:
    - toFQDNs:
        - matchName: example.com
      toPorts:
        - ports:
```

```
    - port: "443"
      protocol: TCP
```

This only works thanks to the `dns` rule from Step 1: Cilium reads the DNS response and remembers the IP addresses for `example.com`.

MAC Step 14

```
kubectl apply -f security/netpol/04-fqdn.yaml
```

Loads the rule.

MAC Step 15

```
kubectl exec -n demo deploy/frontend -- \
  curl -s -m 5 -o /dev/null -w '%{http_code}\n' https://example.com/
```

Expected: `200`.

MAC Step 16

```
kubectl exec -n demo deploy/frontend -- \
  curl -s -m 5 -o /dev/null -w '%{http_code}\n' https://example.org/
```

Expected: `000`. Only the named host is allowed.

6.7 What Still Gets Through

PITFALL

The kubelet keeps checking pods with health checks, even though everything is blocked. Cilium allows traffic from the pod's own node by default. Pods that need the API server, however, have to be allowed explicitly: with `egress` and `toEntities: [kube-apiserver]`. Without this rule, operators fail silently.

6.8 How to Apply This to Real Namespaces

Never lock down a whole cluster in one go. Work namespace by namespace:

1. Let the service run under load.
2. Look in Hubble or Grafana to see which connections actually exist.
3. Write the rules to match.
4. Turn on default deny and watch `--verdict DROPPED`.

The file `01-default-deny.yaml` can serve as a template. `kube-system` stays out of this, because that is where the core services run.

TRY IT YOURSELF

Secure the `falco-lab` namespace from Chapter 7 once you have it: only DNS and egress to `dl-cdn.alpinelinux.org`. Then `apk add` keeps working, but nothing else does.

WHAT YOU LEARNED

- As soon as a policy selects a pod, default deny applies to that direction.
- A connection needs both sides: egress at the sender, ingress at the receiver.
- `hubble observe --verdict DROPPED` replaces guessing.
- Cilium also understands HTTP and hostnames, not just ports.
- A timeout (`000`) and `403` mean different things: a block at the network level or at the application level.

CHAPTER 7

Falco: Detecting Suspicious Activity

Falco watches system calls on the node and reports when a container behaves suspiciously.

Time: 60 minutes **Requires:** Chapter 4

7.1 The Alarm System in the Kernel

So far you have been preventing things. But what if someone gets through anyway? Falco sits in the kernel via eBPF and sees every system call of every container. It compares them against rules and reports matches.

Falco sees	Falco does not see
A shell starts in the container	The content of encrypted data
A process reads `/etc/shadow`	Whether an action is allowed from a business point of view
A new program is executed	What happens outside the node

WHY IT MATTERS

Falco blocks nothing; it only reports. That is why it complements Cilium and PSA: whatever prevention lets through, Falco sees. A rule that never fires cannot disturb anything. A noisy rule, on the other hand, makes real alerts get lost. Fine-tuning is therefore the real work.

7.2 Installing

Falco writes its alerts as JSON to standard output. For now, you read them with `kubectl logs`. In Chapter 10, Alloy collects them automatically and ships them to Loki. So you do not need any extra add-on, and you save RAM.

FILE `falco/values.yaml`

```yaml
# Print lines immediately, do not buffer
tty: true

driver:
  # Modern eBPF driver, no kernel module needed
  kind: modern_ebpf
```

```
falco:
  json_output: true
  json_include_output_property: true

resources:
  requests:
    cpu: 100m
    memory: 256Mi
  limits:
    memory: 768Mi

customRules:
  eigene-regeln.yaml: |-
    - rule: Paketmanager im Container
      desc: Someone is installing software in a running container
      condition: >
        spawned_process and container
        and proc.name in (apt, apt-get, apk, dnf, yum)
      output: >
        Package manager started (user=%user.name cmd=%proc.cmdline
        pod=%k8s.pod.name ns=%k8s.ns.name
        image=%container.image.repository)
      priority: WARNING
      tags: [container, mitre_persistence]
```

`modern_ebpf` runs on your kernel 6.8 without compiling anything. The chart already knows the k3s socket `/run/k3s/containerd/containerd.sock`, so you do not need to add it. That is how Falco gets container names and image info.

MAC — Step 1

```
helm repo add falcosecurity https://falcosecurity.github.io/charts
helm repo update
```

Adds the repository.

MAC — Step 2

```
helm upgrade --install falco falcosecurity/falco \
  -n falco --version 9.2.2 \
  -f falco/values.yaml --wait --timeout 5m
```

The `falco` namespace has existed since Chapter 4 with the `privileged` level. If you had not created it, the cluster-wide default would reject Falco.

MAC — Step 3

```
kubectl logs -n falco ds/falco -c falco --tail=20
```

Checks the startup. You should see that rules were loaded and the eBPF probe is active. Errors in your own rules also show up here.

7.3 Simulating an Attack

You need a victim that runs as root. That is not possible in `demo` because of `restricted`. This difference is already the first lesson.

MAC Step 4

```
kubectl create namespace falco-lab
```

A namespace with the `baseline` default, where root is allowed.

MAC Step 5

```
kubectl run opfer -n falco-lab --image=nginx:1.29-alpine
kubectl wait pod/opfer -n falco-lab --for=condition=Ready
```

Starts the victim and waits until it is ready.

Open a second terminal and follow Falco live:

MAC Step 6

```
kubectl logs -n falco ds/falco -c falco -f --tail=0
```

`-f` follows the log, `--tail=0` shows only new lines. Leave this terminal open.

Back in the first terminal, you perform typical attacker actions:

MAC Step 7

```
kubectl exec -it opfer -n falco-lab -- sh
```

Opens a shell in the container. That alone triggers the `Terminal shell in container` rule.

POD Step 8

```
cat /etc/shadow
```

Reads the password hashes. Rule: `Read sensitive file untrusted`.

POD Step 9

```
apk add --no-cache curl
```

Installs software. This triggers your own rule `Paketmanager im Container`.

POD Step 10

```
exit
```

Leaves the container.

WHAT YOU SHOULD SEE

JSON lines with `rule`, `priority` and `output_fields` appear in the second terminal. You see the pod, namespace, image and the exact command line. A missing line for `Paketmanager` usually points to a mistake in `eigene-regeln.yaml`; in that case, check the startup logs.

7.4 Silencing False Positives

Imagine your team opens shells in `falco-lab` every day. Then the `Terminal shell in container` rule turns into noise. You extend it with an exception instead of turning it off.

In `falco/values.yaml`, add a second file under `customRules`:

FILE `falco/values.yaml (excerpt)`

```yaml
customRules:
  eigene-regeln.yaml: |-
    # ... as above ...
  ausnahmen.yaml: |-
    - rule: Terminal shell in container
      condition: and not k8s.ns.name = "falco-lab"
      override:
        condition: append
```

`override: condition: append` appends your text to the existing condition. That way the original rule stays intact, and you keep getting updates. The files are loaded in alphabetical order after the default rules.

MAC Step 11

```
helm upgrade falco falcosecurity/falco \
  -n falco --version 9.2.2 -f falco/values.yaml --wait
```

Loads the new rules.

MAC Step 12

```
kubectl exec opfer -n falco-lab -it -- sh -c 'echo test'
```

Falco should stay quiet now. In all other namespaces it keeps reporting shells.

PITFALL

Keep exceptions as narrow as possible. An exception for a whole namespace is fine for a lab. In production, a combination of image and user is the better choice. Otherwise you build the attacker a blind spot.

TRY IT YOURSELF

Write a rule that fires when a container opens a connection on port 4444, the typical port for reverse shells. Hint: look at the `outbound` macro and the `fd.sport` field in the Falco docs. Test with `nc` in the `opfer` pod.

WHAT YOU LEARNED

- Falco sees system calls, but blocks nothing.
- JSON on standard output is enough; Chapter 10 collects it centrally.
- Fine-tuning is the real work, because noisy rules hide real alerts.
- With `override: append` you extend rules without overwriting them.

CHAPTER 8

Trivy Operator: Scanning Regularly

The cluster checks itself for known vulnerabilities and misconfigurations, and you analyze the reports.

Time: 45 minutes **Requires:** Chapter 4

8.1 Finding Known Weaknesses

An image that is clean today can be vulnerable tomorrow, because a new flaw becomes known. So scanning once is not enough. The Trivy Operator watches the cluster and scans everything that runs. It stores the results as Kubernetes objects.

Report	Short name	Content
`VulnerabilityReport`	`vuln`	Known CVEs in images
`ConfigAuditReport`	`configaudit`	Insecure pod settings
`ExposedSecretReport`	`exposedsecret`	Passwords stored in the image
`RbacAssessmentReport`	`rbacassessment`	Roles that are too broad
`InfraAssessmentReport`	`infraassessment`	Node settings
`ClusterComplianceReport`	`compliance`	Checks against standards such as CIS

WHY IT MATTERS

Because the reports are Kubernetes objects, RBAC, `kubectl` and Prometheus apply to them. You don't need a separate scan server, and each team sees the reports for its own namespace.

8.2 Installing

FILE `trivy/values.yaml`

```yaml
# Chapter 10 turns this on once Prometheus is running.
serviceMonitor:
  enabled: false

trivy:
  # Only show vulnerabilities that have a fix
  ignoreUnfixed: true
  severity: HIGH,CRITICAL
```

```
operator:
  # Only one scan at a time, to spare the node
  scanJobsConcurrentLimit: 1
  # Recreate reports after 24 hours
  scannerReportTTL: 24h
```

`ignoreUnfixed` hides findings you can't do anything about right now. `scanJobsConcurrentLimit: 1` matters on 8 GB of RAM: the default of 10 could overload your node. `scannerReportTTL` makes sure a new scan runs every day.

MAC Step 1

```
helm repo add aqua https://aquasecurity.github.io/helm-charts/
helm repo update
```

Adds the repository.

MAC Step 2

```
helm upgrade --install trivy-operator aqua/trivy-operator \
  -n trivy-system --version 0.38.1 \
  -f trivy/values.yaml --wait --timeout 5m
```

Installs the operator into the prepared namespace. `trivy-system` has had the `privileged` level since Chapter 4, because the node collector checks host files.

MAC Step 3

```
kubectl get jobs -n trivy-system
```

The operator creates a short-lived scan job for each workload. On a fresh start it takes a while until all of them are done, and the vulnerability database is downloaded once.

8.3 Letting It Find an Old Image

MAC Step 4

```
kubectl create deployment altlast -n falco-lab --image=nginx:1.21.6
```

Starts an image from 2022. It contains plenty of known vulnerabilities. The `falco-lab` namespace allows root.

MAC Step 5

```
kubectl get vuln -n falco-lab -o wide
```

After a few minutes a report appears. The columns show the count per severity. As long as the list is empty, the scan is still running.

MAC Step 6

```
kubectl get vuln -n falco-lab -o json | jq -r '
  .items[].report.vulnerabilities[]
  | select(.severity == "CRITICAL")
  | "\(.vulnerabilityID)  \(.resource)  fix: \(.fixedVersion)"
' | sort -u | head -20
```

Shows the critical vulnerabilities with the package and the version that fixes them.

WHAT YOU SHOULD SEE

You see a list of CVE numbers, for example for `openssl` or `zlib`, each with a `fix:` version. The exact list changes with the database.

TRY IT YOURSELF

Update the deployment: `kubectl set image deploy/altlast -n falco-lab altlast=nginx:1.29-alpine`. How many critical vulnerabilities does the new report show? In practice this is the most common way to close vulnerabilities: don't patch, roll out a new image.

8.4 Finding Misconfigurations

MAC Step 7

```
kubectl get configaudit -A -o wide
```

Shows the number of findings per workload. Your pod in `demo` should score much better than `opfer` in `falco-lab`, because it meets `restricted`.

MAC Step 8

```
kubectl get configaudit -n falco-lab -o json | jq -r '
  .items[].report.checks[]
  | select(.success == false)
  | "\(.severity)  \(.checkID)  \(.title)"
' | sort -u
```

Lists all failed checks. Many of them match exactly the rules you know from Chapter 4.

MAC Step 9

```
kubectl get rbacassessment -A -o wide
```

Rates your roles. Here you see how Trivy assesses the permissions from Chapter 3.

MAC Step 10

```
kubectl get exposedsecret -A
```

Shows secrets that someone baked into an image. This list should be empty.

PITFALL

The infra checks often expect the standard paths of `kubeadm`. In k3s the API server runs inside the k3s process, so `infraassessment` produces some false positives. Check such findings before you act, and compare them with the k3s hardening guide.

8.5 The Rule That Ties It All Together

Chapter 5 and this chapter fit together: Trivy reports that an image is vulnerable, and Kyverno can use an `ImageValidatingPolicy` to require that only verified images start. That leads to signed images and makes a good next learning project.

WHAT YOU LEARNED

- Scanning is not a one-time event, because new vulnerabilities affect old images too.
- Trivy stores reports as Kubernetes objects, so RBAC and `kubectl` apply.
- On small nodes, `scanJobsConcurrentLimit` limits the load.
- You usually close vulnerabilities with a new image, not with a patch inside the container.

CHAPTER 9

Audit Logs: Who Did What?

The API server records every important request, and you analyze the log.

Time: 30 minutes **Requires:** Chapter 4

9.1 The API Server's Security Camera

Every change in the cluster goes through the API server. That makes it the perfect place for a log. After an incident, the audit log answers the key questions: who performed which action on which object, when, and from where?

In k3s the API server runs inside the k3s process, not as a pod. So the audit log is a file on the host. For now you read it directly there; in Chapter 10, Alloy ships it to Loki.

9.2 The Four Log Levels

Level	What gets logged
`None`	Nothing
`Metadata`	Who, what, when, result. No content.
`Request`	Also the content of the request
`RequestResponse`	Also the content of the response

WHY IT MATTERS

Never use `Request` or `RequestResponse` for Secrets. Otherwise the passwords end up in plain text in the log, and your log becomes a risk itself. The first matching rule wins, so the order in the file matters.

9.3 Writing the Audit Policy

FILE `security/audit-policy.yaml`

```yaml
apiVersion: audit.k8s.io/v1
kind: Policy
omitStages:
  - RequestReceived
rules:
```

```
  # Drop the noise: health checks and metrics
  - level: None
    nonResourceURLs:
      - /healthz*
      - /readyz*
      - /livez*
      - /metrics
  # Events and leader leases fire constantly
  - level: None
    resources:
      - group: ""
        resources: [events]
      - group: coordination.k8s.io
        resources: [leases]
  # Never write the content of Secrets to the log
  - level: Metadata
    resources:
      - group: ""
        resources: [secrets, configmaps]
      - group: authentication.k8s.io
        resources: [tokenreviews]
  # Record permission changes in full
  - level: RequestResponse
    verbs: [create, update, patch, delete]
    resources:
      - group: rbac.authorization.k8s.io
  # Everything else as metadata only
  - level: Metadata
```

Without the two `None` rules at the top, the log fills up with worthless entries. `omitStages` drops the first stage of each request; otherwise every action would appear twice in the log.

9.4 Enabling It on the Node

MAC — Step 1

```
scp security/audit-policy.yaml \
  <user>@172.16.0.136:/tmp/audit.yaml
```

Copies the policy to the node.

Now open `k3s/config.yaml` and uncomment the audit lines as well. The `admission-control` line has been active since Chapter 4.

FILE `k3s/config.yaml (excerpt)`

```
kube-apiserver-arg:
  - "admission-control-config-file=/var/lib/rancher/k3s/server/psa.yaml"
  - "audit-policy-file=/var/lib/rancher/k3s/server/audit.yaml"
  - "audit-log-path=/var/lib/rancher/k3s/server/logs/audit.log"
  - "audit-log-maxage=30"
  - "audit-log-maxbackup=10"
  - "audit-log-maxsize=100"
```

`maxsize` rotates the file at 100 MB, `maxbackup` keeps ten old files, `maxage` deletes them after 30 days. That limits the disk usage to about 1.1 GB.

MAC Step 2

```
scp k3s/config.yaml <user>@172.16.0.136:/tmp/config.yaml
ssh <user>@172.16.0.136
```

Copies the changed configuration to the node and opens the session.

NODE Step 3

```
sudo install -m 600 -o root -g root /tmp/audit.yaml \
  /var/lib/rancher/k3s/server/audit.yaml
```

The policy must be in place before the restart, otherwise the API server won't start.

NODE Step 4

```
sudo install -m 600 -o root -g root /tmp/config.yaml \
  /etc/rancher/k3s/config.yaml
rm /tmp/audit.yaml /tmp/config.yaml
```

Puts the new configuration in place and cleans up `/tmp`.

NODE Step 5

```
sudo systemctl restart k3s
```

k3s reads `config.yaml` only at startup. The pods keep running during the restart.

NODE Step 6

```
sudo tail -n 1 /var/lib/rancher/k3s/server/logs/audit.log | jq .
```

Shows the last entry in readable form. Read it all the way through once: `user`, `verb`, `objectRef`, `sourceIPs`, `responseStatus`.

PITFALL

If k3s no longer starts, the policy file is usually to blame. `sudo journalctl -u k3s -n 50` names the error. If all else fails, comment out the lines in `/etc/rancher/k3s/config.yaml` and restart.

9.5 Analyzing the Log

Keep the SSH session open and open a second terminal on your laptop.

MAC Step 7

```
kubectl create secret generic audit-test \
  --from-literal=wert=streng-geheim
```

Creates a test Secret that you will look for in the log in a moment.

MAC Step 8

```
kubectl get secret audit-test -o yaml
```

Reads it, so that a read access shows up in the log too.

NODE Step 9

```
sudo jq -c 'select(.objectRef.name == "audit-test")
  | {verb, user: .user.username, code: .responseStatus.code}' \
  /var/lib/rancher/k3s/server/logs/audit.log
```

`jq` reads the log as a stream of JSON objects. `select` filters, and the second stage builds a short line.

WHAT YOU SHOULD SEE

You see entries with `create` and `get` and your admin user.

NODE Step 10

```
sudo grep -c streng-geheim \
  /var/lib/rancher/k3s/server/logs/audit.log
```

Must print `0` : the secret value is **not** in the log. That is exactly what the `Metadata` rule did.

MAC Step 11

```
kubectl delete secret audit-test
```

Cleans up the test object.

9.6 Useful Queries

Who besides system components has read Secrets?

NODE Step 12

```
sudo jq -c 'select(.objectRef.resource == "secrets"
    and (.verb == "get" or .verb == "list")
    and (.user.username | startswith("system:") | not))
  | {verb, user: .user.username, name: .objectRef.name}' \
  /var/lib/rancher/k3s/server/logs/audit.log
```

The most important query for everyday use. In Chapter 10 you rewrite it in LogQL.

Which requests were denied?

NODE Step 13

```
sudo jq -c 'select(.responseStatus.code == 403)
  | {verb, user: .user.username, res: .objectRef.resource}' \
  /var/lib/rancher/k3s/server/logs/audit.log
```

Denied requests often point to misconfigurations, but sometimes also to an attacker looking around.

NODE

Step 14

```
exit
```

Ends the SSH session.

TRY IT YOURSELF

Run `kubectl get secrets -A --as=system:anonymous`. What does the cluster answer, and what shows up in the audit log afterwards? Then start a plain `nginx` pod in a new namespace and search the log for the annotation `pod-security.kubernetes.io/audit-violations`. That is the PSA audit mode from Chapter 4.

WHAT YOU LEARNED

- After an incident, the audit log answers: who, what, when, from where.
- The first matching rule wins, so the `None` rules go at the top.
- Secrets belong in the log only at the `Metadata` level.
- In k3s the audit log is a host file. As long as it only lives there, an attacker can delete it.

CHAPTER 10

A Central View: Grafana and Loki

All layers report to one place: audit log, Falco, Trivy and Hubble end up in Grafana.

Time: 90 minutes **Requires:** Chapters 7, 8 and 9

10.1 What You Build

So far, all your findings are scattered: in the audit log on the node, in the Falco logs, in Trivy objects. Now you bring them together.

Component	Job
SeaweedFS	S3 storage for logs and traces, from your repo
Prometheus	Stores metrics
Loki	Stores logs
Alloy	Collects logs on the node
Grafana	Shows everything in one place

WHY IT MATTERS

Logs that only live on the node can be deleted by an attacker. Once they reach Loki, they sit in object storage, out of the attacker's reach. That is exactly why visibility is a security measure in itself.

WHAT ABOUT THANOS?

Your repo also contains Thanos for long-term metric storage. You don't need it for this book, and leaving it out saves about 0.5 GB of RAM. If you want to add it later, follow the repo's README.

10.2 Starting SeaweedFS

The `monitoring` namespace has existed since Chapter 4 with the `privileged` level.

MAC Step 1

```
./seaweedfs/create-secrets.sh
```

Your script from the repo. It creates the `s3-config` Secret with the first account, which you will add more accounts to in a moment.

PITFALL

`seaweedfs/seaweedfs.yaml` limits storage with `-volume.max=16` to 16 × 256 MB, so about 4 GB. That gets tight for Loki and Tempo. Raise the value in the file to `-volume.max=32` before you apply it. Your disk can handle the 8 GB.

MAC Step 2

```
kubectl apply -f seaweedfs/seaweedfs.yaml
```

Creates the PVC, Deployment and Service for the S3 storage.

MAC Step 3

```
kubectl rollout status deploy/s3 -n monitoring
```

Waits until the pod is ready. `apply` returns immediately; this command does not.

MAC Step 4

```
kubectl exec -n monitoring deploy/s3 -- sh -c \
  "echo 's3.bucket.create -name loki' | weed shell -master=localhost:9333"
```

Creates the `loki` bucket through the SeaweedFS admin shell.

10.3 A Dedicated S3 Account for Loki

Each service gets its own account with access only to its own bucket. If a service is compromised, only that bucket is affected.

MAC Step 5

```
LOKI_ACCESS=$(openssl rand -hex 10)
LOKI_SECRET=$(openssl rand -hex 20)
```

Generates two random values and stores them in shell variables. They appear neither on screen nor in a file.

MAC Step 6

```
ALT=$(kubectl get secret s3-config -n monitoring \
  -o jsonpath='{.data.s3\.json}' | base64 -d)
```

Reads the existing account list. Kubernetes stores Secrets Base64-encoded; `base64 -d` makes them readable again. The backslash in `s3\.json` is needed because the dot would otherwise count as a path separator.

MAC Step 7

```
NEU=$(printf '%s' "$ALT" | jq \
  --arg a "$LOKI_ACCESS" --arg s "$LOKI_SECRET" \
  '.identities += [{
     name: "loki",
     credentials: [{accessKey: $a, secretKey: $s}],
     actions: ["Read:loki","Write:loki","List:loki","Tagging:loki"]
   }]')
```

`jq` appends a new account to the list. `--arg` passes the values safely instead of pasting them into the text. `actions` restricts the account to the `loki` bucket.

MAC Step 8

```
printf '%s' "$NEU" | kubectl create secret generic s3-config \
  -n monitoring --from-file=s3.json=/dev/stdin \
  --dry-run=client -o yaml | kubectl apply -f -
```

Builds a new Secret locally (`--dry-run=client -o yaml`) and applies it. The data goes through standard input so that it doesn't show up in the process list.

MAC Step 9

```
kubectl rollout restart deploy/s3 -n monitoring
kubectl rollout status deploy/s3 -n monitoring
```

SeaweedFS reads the accounts only at startup, so you have to restart it.

MAC Step 10

```
kubectl create secret generic loki-s3 -n monitoring \
  --from-literal=S3_ACCESS_KEY="$LOKI_ACCESS" \
  --from-literal=S3_SECRET_KEY="$LOKI_SECRET"
```

Stores the credentials for Loki. The key names later become environment variables.

MAC Step 11

```
unset LOKI_ACCESS LOKI_SECRET ALT NEU
```

Removes the secrets from your shell.

10.4 Installing Prometheus and Grafana

First create a values file that adapts the stack to k3s:

FILE monitoring/kps-k3s-values.yaml

```
grafana:
  admin:
    existingSecret: grafana-admin
    userKey: admin-user
    passwordKey: admin-password
  additionalDataSources:
```

```
    - name: Loki
      uid: loki
      type: loki
      access: proxy
      url: http://loki.monitoring.svc.cluster.local:3100
      editable: false
    - name: Tempo
      uid: tempo
      type: tempo
      access: proxy
      url: http://tempo.monitoring.svc.cluster.local:3200
      editable: false
      jsonData:
        tracesToLogsV2:
          datasourceUid: loki

# Cilium replaces kube-proxy, so there is nothing to monitor.
kubeProxy:
  enabled: false
# In k3s these run inside the k3s process, not as separate pods.
kubeControllerManager:
  enabled: false
kubeScheduler:
  enabled: false
kubeEtcd:
  enabled: false

prometheus:
  prometheusSpec:
    retention: 2d
    # Also pick up ServiceMonitors without the label "release: kps".
    serviceMonitorSelectorNilUsesHelmValues: false
    podMonitorSelectorNilUsesHelmValues: false
    ruleSelectorNilUsesHelmValues: false
```

The last three values are important: by default, Prometheus only looks at objects from its own Helm release. With `false` it also finds the ServiceMonitors from Trivy and Cilium and your rules. The Tempo data source is already in place; Tempo itself comes in Chapter 11.

MAC

Step 12

```
kubectl create secret generic grafana-admin -n monitoring \
  --from-literal=admin-user=admin \
  --from-literal=admin-password="$(openssl rand -hex 16)"
```

Creates a random Grafana password. It is not in any file, only in the cluster.

MAC

Step 13

```
helm repo add prometheus-community \
  https://prometheus-community.github.io/helm-charts
helm repo update
```

If the repository is already added, Helm just tells you so. `update` fetches the current version lists.

MAC Step 14

```
helm upgrade --install kps \
  prometheus-community/kube-prometheus-stack \
  -n monitoring --version 92.18.0 \
  -f monitoring/kps-k3s-values.yaml \
  --wait --timeout 10m
```

Installs Prometheus, Alertmanager, Grafana and the exporters. The version matches your README.

10.5 Installing Loki

The Loki chart has moved to the Grafana community. The old `grafana/loki` is stuck at version 7.5.0 and is no longer kept up to date.

FILE monitoring/loki-values.yaml

```
deploymentMode: Monolithic

loki:
  auth_enabled: false
  commonConfig:
    replication_factor: 1
  schemaConfig:
    configs:
      - from: "2026-01-01"
        store: tsdb
        object_store: s3
        schema: v13
        index:
          prefix: loki_index_
          period: 24h
  storage:
    type: s3
    bucketNames:
      chunks: loki
      ruler: loki
      admin: loki
    s3:
      endpoint: http://s3.monitoring.svc.cluster.local:8333
      region: us-east-1
      # Loki replaces these from the environment variables at startup.
      accessKeyId: ${S3_ACCESS_KEY}
      secretAccessKey: ${S3_SECRET_KEY}
      s3ForcePathStyle: true
      insecure: true
  compactor:
    retention_enabled: true
    delete_request_store: s3
  limits_config:
    retention_period: 168h

singleBinary:
  replicas: 1
  extraEnvFrom:
    - secretRef:
        name: loki-s3
  persistence:
    size: 5Gi
    storageClass: local-path
```

```
  resources:
    requests:
      cpu: 100m
      memory: 256Mi
    limits:
      memory: 1Gi

# On a small node, turning these off saves a lot of RAM.
gateway:
  enabled: false
chunksCache:
  enabled: false
resultsCache:
  enabled: false
lokiCanary:
  enabled: false
test:
  enabled: false
backend:
  replicas: 0
read:
  replicas: 0
write:
  replicas: 0
```

`replication_factor: 1` is mandatory with only one instance, otherwise requests fail. `s3ForcePathStyle` is required for SeaweedFS, and `retention_period: 168h` keeps logs for seven days. With `${...}` no secrets end up in the file. The chart sets `-config.expand-env=true` itself.

MAC

Step 15

```
helm repo add grafana-community \
  https://grafana-community.github.io/helm-charts
helm repo update
```

The new home of the charts for Loki and Tempo.

MAC

Step 16

```
helm upgrade --install loki grafana-community/loki \
  -n monitoring --version 21.0.0 \
  -f monitoring/loki-values.yaml --wait --timeout 5m
```

Installs Loki as a single process. According to the docs, that is enough for a few tens of GB per day.

MAC

Step 17

```
kubectl logs -n monitoring statefulset/loki --tail=20
```

Checks the startup. If there are S3 problems, you see messages like `AccessDenied` here, which means the account is wrong.

10.6 Alloy Collects the Logs

Alloy runs as a DaemonSet, so once per node. It reads the container logs under `/var/log/pods` and your audit log, and sends both to Loki. You keep the configuration separate from the Helm values so that you can change it easily.

FILE monitoring/config.alloy

```
// Find pods on this node
discovery.kubernetes "pods" {
  role = "pod"
  selectors {
    role  = "pod"
    field = "spec.nodeName=" + sys.env("HOSTNAME")
  }
}

// Set labels and build the path to the log file
discovery.relabel "pods" {
  targets = discovery.kubernetes.pods.targets

  rule {
    source_labels = ["__meta_kubernetes_namespace"]
    target_label  = "namespace"
  }
  rule {
    source_labels = ["__meta_kubernetes_pod_name"]
    target_label  = "pod"
  }
  rule {
    source_labels = ["__meta_kubernetes_pod_container_name"]
    target_label  = "container"
  }
  rule {
    source_labels = ["__meta_kubernetes_pod_uid",
                     "__meta_kubernetes_pod_container_name"]
    separator     = "/"
    replacement   = "/var/log/pods/*$1/*.log"
    target_label  = "__path__"
  }
}

local.file_match "pods" {
  path_targets = discovery.relabel.pods.output
}

loki.source.file "pods" {
  targets    = local.file_match.pods.targets
  forward_to = [loki.process.pods.receiver]
}

// Unpack the containerd CRI format
loki.process "pods" {
  stage.cri {}
  forward_to = [loki.write.default.receiver]
}

// API server audit log from Chapter 9
local.file_match "audit" {
  path_targets = [{
    "__path__" = "/k3s-logs/audit.log",
    "job"      = "audit",
```

```
  }]
}

loki.source.file "audit" {
  targets    = local.file_match.audit.targets
  forward_to = [loki.write.default.receiver]
}

loki.write "default" {
  endpoint {
    url = "http://loki.monitoring.svc.cluster.local:3100/loki/api/v1/push"
  }
}
```

Read the file from bottom to top: `loki.write` is the destination, and each block passes data on with `forward_to`.

FILE `monitoring/alloy-values.yaml`

```
alloy:
  # The audit log belongs to root and is readable only by root.
  securityContext:
    runAsUser: 0
    runAsGroup: 0
  configMap:
    create: false
    name: alloy-config
    key: config.alloy
  mounts:
    varlog: true
    extra:
      - name: k3s-logs
        mountPath: /k3s-logs
        readOnly: true

controller:
  type: daemonset
  volumes:
    extra:
      - name: k3s-logs
        hostPath:
          path: /var/lib/rancher/k3s/server/logs
          type: DirectoryOrCreate
```

Alloy has to run as root here, and that is a deliberate trade-off: the pod has read access to the host, but in return it mounts the audit directory read-only. The chart also gives the pod the permissions it needs for `discovery.kubernetes`.

MAC Step 18

```
kubectl create configmap alloy-config -n monitoring \
  --from-file=config.alloy=monitoring/config.alloy
```

Stores the configuration as a ConfigMap. For later changes, add `--dry-run=client -o yaml | kubectl apply -f -`.

MAC Step 19

```
helm repo add grafana https://grafana.github.io/helm-charts
helm repo update
```

Alloy stays in Grafana's main repository.

MAC Step 20

```
helm upgrade --install alloy grafana/alloy \
  -n monitoring --version 1.15.0 \
  -f monitoring/alloy-values.yaml --wait
```

Starts Alloy as a DaemonSet.

MAC Step 21

```
kubectl logs -n monitoring ds/alloy -c alloy --tail=20
```

Syntax errors in `config.alloy` show up here with a line number.

10.7 Connecting Hubble and Trivy

Both services are already running. Now that Prometheus has brought along the ServiceMonitor CRDs, they can deliver metrics.

FILE `cilium/metrics-values.yaml`

```
prometheus:
  enabled: true
  serviceMonitor:
    enabled: true
operator:
  prometheus:
    enabled: true
    serviceMonitor:
      enabled: true
hubble:
  metrics:
    enabled:
      - dns
      - drop
      - tcp
      - flow
      - policy:sourceContext=workload-name;destinationContext=workload-name
    serviceMonitor:
      enabled: true
```

MAC Step 22

```
helm upgrade cilium oci://quay.io/cilium/charts/cilium \
  --version 1.20.4 -n kube-system \
  -f cilium/values.yaml -f cilium/metrics-values.yaml --wait
```

Merges both files. The agents restart in the process; existing connections usually keep running.

In `trivy/values.yaml`, change the line under `serviceMonitor` to `enabled: true`.

MAC Step 23

```
helm upgrade trivy-operator aquа/trivy-operator \
  -n trivy-system --version 0.38.1 \
  -f trivy/values.yaml --wait
```

Creates the ServiceMonitor for Trivy.

Falco already writes to standard output, so there is nothing more to do for it. However, the default deny from Chapter 6 locks Prometheus out of `demo`. If your application should deliver metrics, you allow that explicitly:

FILE `security/netpol/05-prometheus.yaml`

```
apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
metadata:
  name: prometheus-darf-scrapen
  namespace: demo
spec:
  endpointSelector: {}
  ingress:
    - fromEndpoints:
        - matchLabels:
            io.kubernetes.pod.namespace: monitoring
            app.kubernetes.io/name: prometheus
```

With `io.kubernetes.pod.namespace` you select pods from another namespace. Only Prometheus is meant here, not the whole namespace.

MAC Step 24

```
kubectl apply -f security/netpol/05-prometheus.yaml
```

Loads the rule.

10.8 Opening Grafana

MAC Step 25

```
kubectl get secret grafana-admin -n monitoring \
  -o jsonpath='{.data.admin-password}' | base64 -d; echo
```

Shows the Grafana password once in the terminal.

MAC Step 26

```
kubectl port-forward -n monitoring svc/kps-grafana 3000:80
```

Grafana is now available at http://localhost:3000. Log in as `admin`.

Open **Explore**, choose the **Loki** data source and enter this query:

```
{namespace="kube-system", container="coredns"}
```

WHAT YOU SHOULD SEE

The CoreDNS log lines appear. If you switch to the **Prometheus** data source and query `hubble_flows_processed_total`, you see the Hubble metrics.

10.9 Querying Audit Logs

Now you rewrite the `jq` queries from Chapter 9 in LogQL. `| json` parses each line, and nested fields like `objectRef.name` become `objectRef_name`.

Who besides system components has read Secrets?

```
{job="audit"} | json
  | objectRef_resource="secrets"
  | verb=~"get|list"
  | user_username!~"system:.*"
```

Which requests were denied?

```
{job="audit"} | json | responseStatus_code="403"
```

Who changed RBAC rules?

```
{job="audit"} | json
  | objectRef_apiGroup="rbac.authorization.k8s.io"
  | verb=~"create|update|patch|delete"
```

10.10 Querying Falco Alerts

`line_format` shows only the rule name:

```
{namespace="falco", container="falco"} | json | line_format "{{.rule}}"
```

Only the serious alerts:

```
{namespace="falco", container="falco"}
  | json
  | priority =~ "Warning|Error|Critical|Alert|Emergency"
```

Counted by rule:

```
sum by (rule) (
  count_over_time(
    {namespace="falco", container="falco"} | json | rule != "" [1h]
  )
)
```

10.11 Querying Trivy Results

Choose the **Prometheus** data source. First check under **Explore** whether `up{job=~".*trivy.*"}` returns the value `1`.

```
sum by (severity) (trivy_image_vulnerabilities)
```

```
topk(5, sum by (namespace, image_repository)
  (trivy_image_vulnerabilities{severity="Critical"}))
sum by (severity) (trivy_resource_configaudits)
```

TRY IT YOURSELF

Write a LogQL query that shows all log lines containing the word `error` from the `monitoring` namespace. Hint: you append filters to the label selector with `|=`.

WHAT YOU LEARNED

- An attacker can't simply delete logs that live outside the node.
- Each service gets its own S3 account, so a breach stays contained.
- Prometheus only picks up other releases' ServiceMonitors once you allow it.
- All layers now end up in one place and can be queried together.

CHAPTER 11

Tempo: Following a Request

You store traces in Tempo and see what happens during an API request, all the way to the check by Kyverno.

Time: 60 minutes **Requires:** Chapter 10

11.1 Three Kinds of Signals

Signal	Answers	Storage
Metrics	How much, how often?	Prometheus
Logs	What exactly happened?	Loki
Traces	Which path did a request take, and where did the time go?	Tempo

A trace consists of **spans**. Each span is one unit of work with a start and a duration.

WHAT TEMPO DOES FOR SECURITY

Tempo is not a security tool in the strict sense, and applications in your cluster only deliver traces once you instrument them. It is still useful: the API server can deliver its requests as traces. You then see, for example, how long the Kyverno webhook holds up a request. This helps with outages caused by admission webhooks, and later, in your own applications, it shows which path a suspicious request took.

11.2 Bucket and S3 Account

You know the pattern from Chapter 10.

MAC Step 1

```
kubectl exec -n monitoring deploy/s3 -- sh -c \
  "echo 's3.bucket.create -name tempo' | weed shell -master=localhost:9333"
```

Creates the bucket `tempo`.

MAC Step 2

```
TEMPO_ACCESS=$(openssl rand -hex 10)
TEMPO_SECRET=$(openssl rand -hex 20)
ALT=$(kubectl get secret s3-config -n monitoring \
  -o jsonpath='{.data.s3\.json}' | base64 -d)
```

Generates the credentials and reads the account list.

MAC Step 3

```
NEU=$(printf '%s' "$ALT" | jq \
  --arg a "$TEMPO_ACCESS" --arg s "$TEMPO_SECRET" \
  '.identities += [{
     name: "tempo",
     credentials: [{accessKey: $a, secretKey: $s}],
     actions: ["Read:tempo","Write:tempo","List:tempo","Tagging:tempo"]
   }]')
```

Appends the account. Tempo may only access its own bucket.

MAC Step 4

```
printf '%s' "$NEU" | kubectl create secret generic s3-config \
  -n monitoring --from-file=s3.json=/dev/stdin \
  --dry-run=client -o yaml | kubectl apply -f -
kubectl rollout restart deploy/s3 -n monitoring
kubectl rollout status deploy/s3 -n monitoring
```

Writes the list back and restarts SeaweedFS.

MAC Step 5

```
kubectl create secret generic tempo-s3 -n monitoring \
  --from-literal=S3_ACCESS_KEY="$TEMPO_ACCESS" \
  --from-literal=S3_SECRET_KEY="$TEMPO_SECRET"
unset TEMPO_ACCESS TEMPO_SECRET ALT NEU
```

Stores the credentials and cleans up the shell.

11.3 Installing Tempo

FILE monitoring/tempo-values.yaml

```
tempo:
  retention: 168h
  storage:
    trace:
      backend: s3
      s3:
        bucket: tempo
        endpoint: s3.monitoring.svc.cluster.local:8333
        # Tempo replaces these at startup from the environment variables.
        access_key: ${S3_ACCESS_KEY}
        secret_key: ${S3_SECRET_KEY}
        insecure: true
        forcepathstyle: true
      wal:
```

```
        path: /var/tempo/wal
  extraArgs:
    config.expand-env: "true"
  extraEnvFrom:
    - secretRef:
        name: tempo-s3
  resources:
    requests:
      cpu: 50m
      memory: 256Mi
    limits:
      memory: 768Mi

persistence:
  enabled: true
  size: 2Gi
  storageClassName: local-path
```

Unlike with Loki, you have to set `config.expand-env` yourself here. The endpoint is given without `http://`; `insecure: true` takes care of the protocol. The `wal` lives locally and buffers traces before they move to S3.

MAC — Step 6

```
helm upgrade --install tempo grafana-community/tempo \
  -n monitoring --version 3.2.1 \
  -f monitoring/tempo-values.yaml --wait --timeout 5m
```

Installs Tempo as a single process, which fits your single node.

MAC — Step 7

```
kubectl logs -n monitoring statefulset/tempo --tail=20
```

Checks the startup. `AccessDenied` again points to the account.

11.4 Alloy Accepts Traces

Alloy does more than collect logs. It also accepts traces via OTLP and forwards them to Tempo. That way there is only one collection point for all signals. Append these blocks to `monitoring/config.alloy`:

FILE monitoring/config.alloy (append at the end)

```
// Accept traces via OTLP
otelcol.receiver.otlp "default" {
  grpc {
    endpoint = "0.0.0.0:4317"
  }
  output {
    traces = [otelcol.processor.batch.default.input]
  }
}

// Send in batches, which is easier on the network
```

```
otelcol.processor.batch "default" {
  output {
    traces = [otelcol.exporter.otlp.tempo.input]
  }
}

otelcol.exporter.otlp "tempo" {
  client {
    endpoint = "tempo.monitoring.svc.cluster.local:4317"
    tls {
      insecure = true
    }
  }
}
```

The API server runs on the host and does not know cluster DNS names. It therefore needs a fixed IP. Add this to `monitoring/alloy-values.yaml`:

FILE `monitoring/alloy-values.yaml (add)`

```
alloy:
  # ... as before, plus:
  extraPorts:
    - name: otlp-grpc
      port: 4317
      targetPort: 4317
      protocol: TCP

# New top-level block
service:
  clusterIP: 10.43.0.50
```

`10.43.0.50` is in the service network and is reachable only inside the cluster and on the node, not from your home network. Cilium cannot serve a `hostPort` on `127.0.0.1` in this mode, which is why you take this route.

MAC Step 8

```
kubectl create configmap alloy-config -n monitoring \
  --from-file=config.alloy=monitoring/config.alloy \
  --dry-run=client -o yaml | kubectl apply -f -
```

Updates the ConfigMap.

MAC Step 9

```
kubectl delete svc alloy -n monitoring
helm upgrade alloy grafana/alloy \
  -n monitoring --version 1.15.0 \
  -f monitoring/alloy-values.yaml --wait
```

You cannot change the ClusterIP of a Service. So you delete it, and Helm creates it again with the new IP.

MAC Step 10

```
kubectl get svc alloy -n monitoring
kubectl logs -n monitoring ds/alloy -c alloy --tail=20
```

The `CLUSTER-IP` column shows `10.43.0.50`, and the log shows no errors.

11.5 The First Trace

The test program `telemetrygen` generates synthetic traces.

MAC Step 11

```
kubectl run telemetrygen -n monitoring --rm -it --restart=Never \
  --image=ghcr.io/open-telemetry/opentelemetry-collector-contrib/telemetrygen:v0.164.1 \
  -- traces --otlp-endpoint=alloy.monitoring.svc.cluster.local:4317 \
  --otlp-insecure --traces=5
```

Sends five traces to Alloy. They travel on to Tempo.

In Grafana, open **Explore**, select the **Tempo** data source and the **TraceQL** query type:

```
{ resource.service.name = "telemetrygen" }
```

WHAT YOU SHOULD SEE

You see a list of traces. A click shows the spans as a waterfall. If nothing shows up, wait a minute and check the Alloy logs for connection errors to Tempo.

11.6 The API Server Delivers Traces

FILE `security/tracing.yaml`

```
apiVersion: apiserver.config.k8s.io/v1
kind: TracingConfiguration
endpoint: 10.43.0.50:4317
# 100000 out of a million: every tenth request
samplingRatePerMillion: 100000
```

For the lab, every tenth request is easy to see. In long-term operation you lower the value, because traces cost storage. According to the Kubernetes docs, the connection is unencrypted and cannot do TLS, so it must only run internally.

MAC Step 12

```
scp security/tracing.yaml <user>@172.16.0.136:/tmp/tracing.yaml
```

Copies the file to the node.

In `k3s/config.yaml`, add one line under `kube-apiserver-arg`:

FILE `k3s/config.yaml (add)`

```
  - "tracing-config-file=/var/lib/rancher/k3s/server/tracing.yaml"
```

MAC — Step 13

```
scp k3s/config.yaml <user>@172.16.0.136:/tmp/config.yaml
ssh <user>@172.16.0.136
```

Transfers the configuration and opens the session.

NODE — Step 14

```
sudo install -m 600 -o root -g root /tmp/tracing.yaml \
  /var/lib/rancher/k3s/server/tracing.yaml
sudo install -m 600 -o root -g root /tmp/config.yaml \
  /etc/rancher/k3s/config.yaml
rm /tmp/tracing.yaml /tmp/config.yaml
sudo systemctl restart k3s
```

Puts both files in place and restarts k3s.

NODE — Step 15

```
exit
```

Back to the laptop.

MAC — Step 16

```
kubectl get pods -A > /dev/null
kubectl run trace-test -n falco-lab --image=nginx:1.29-alpine
kubectl delete pod trace-test -n falco-lab
```

Generates API requests. Repeat this a few times, because only every tenth one is recorded.

```
{ resource.service.name = "apiserver" }
```

Open a trace for creating a pod and look at the spans. To search specifically for webhook calls, use this:

```
{ resource.service.name = "apiserver" && name =~ ".*webhook.*" }
```

WHAT YOU SHOULD SEE

You find spans for webhook calls, including the one to Kyverno, with their duration. The exact span names depend on the Kubernetes version. If you find nothing, remove the name filter and search the waterfall.

PITFALL

Traces can contain internal URLs and object names. So treat Tempo like logs: no access from outside. Also, the OTLP port accepts anything without authentication, which means any pod in the cluster could submit traces. In namespaces with default deny this is blocked until you allow it.

11.7 Jumping from Traces to Logs

The data source has `tracesToLogsV2` set from Chapter 10. In a span, click **Logs for this span**, and Grafana opens the matching Loki lines. This takes you from “this request was slow” to “this is what the log said at the time”.

TRY IT YOURSELF

Stop Kyverno with `kubectl scale deploy -n kyverno kyverno-admission-controller --replicas=0` and create a pod in `falco-lab`. What happens, and what does the trace look like? Afterwards, be sure to scale back up to `1`.

WHAT YOU LEARNED

- Metrics count, logs tell, traces show the path.
- Alloy is the shared collection point for logs and traces.
- The API server delivers traces too, and they show you the admission webhooks.
- Services on the host need a fixed IP, because they do not know cluster DNS.

CHAPTER 12

Bringing It All Together

Individual tools become an architecture: one dashboard, alerts and an attack drill across all layers.

Time: 90 minutes **Requires:** Chapters 1 to 11

12.1 The Security Dashboard

A good dashboard answers one question in ten seconds: Is something going on right now? In Grafana, go to **Dashboards > New**, create a new dashboard and build these panels. You learn more this way than by importing a finished JSON file.

Panel	Source	Query
Falco by rule	Loki	See query below
Dropped packets	Prometheus	`sum by (reason) (rate(hubble_drop_total[5m]))`
Critical CVEs	Prometheus	`sum(trivy_image_vulnerabilities{severity="Critical"})`
Misconfigurations	Prometheus	`sum by (severity) (trivy_resource_configaudits)`
Denied API requests	Loki	See query below
Slow webhooks	Tempo	See query below

Falco by rule (panel type **Bar chart**):

```
sum by (rule) (
  count_over_time(
    {namespace="falco", container="falco"}
    | json | priority =~ "Warning|Error|Critical" [1h]
  )
)
```

Denied API requests (panel type **Time series**):

```
sum(
  count_over_time(
    {job="audit"} | json | responseStatus_code="403" [5m]
  )
)
```

Webhook calls over 100 milliseconds (panel type **Table**):

```
{ resource.service.name = "apiserver" && name =~ ".*webhook.*"
  && duration > 100ms }
```

Human access to secrets (panel type **Logs**):

```
{job="audit"} | json
  | objectRef_resource="secrets"
  | user_username !~ "system:.*"
```

The Kyverno status, which you see with `kubectl get polr -A`, fits in here as well. If you want it on the dashboard, set `serviceMonitor.enabled: true` for `admissionController` and `reportsController` in `kyverno/values.yaml`.

WHY IT MATTERS

Export the finished dashboard as JSON to `security/dashboard.json`. Stored in the repo, it survives a rebuild of the VM.

12.2 Alerts

A dashboard only helps if someone looks at it. For the most important cases, you create rules.

FILE `security/alerts.yaml`

```yaml
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: security
  namespace: monitoring
spec:
  groups:
    - name: security
      rules:
        - alert: KritischeSchwachstelle
          expr: sum(trivy_image_vulnerabilities{severity="Critical"}) > 0
          for: 1h
          labels:
            severity: warning
          annotations:
            summary: Images with critical CVEs are running.
        - alert: VieleBlockiertePakete
          expr: sum(rate(hubble_drop_total{reason="POLICY_DENIED"}[5m])) > 1
          for: 10m
          labels:
            severity: warning
          annotations:
            summary: Unusually many blocked connections.
```

`for: 1h` avoids alerts on short-lived spikes. Thanks to `ruleSelectorNilUsesHelmValues: false` from Chapter 10, Prometheus finds the rule.

MAC Step 1

```
kubectl apply -f security/alerts.yaml
```

Loads the rules.

MAC Step 2

```
kubectl port-forward -n monitoring \
  svc/kps-kube-prometheus-stack-prometheus 9090
```

At http://localhost:9090/alerts you see the state of the alerts.

PITFALL

The reason value `POLICY_DENIED` is a common value of `hubble_drop_total`, but it can differ between versions. Check the real values with `sum by (reason) (hubble_drop_total)` before you rely on the alert.

For Falco, you create a Grafana alert: open the panel **Falco by rule**, choose **More > New alert rule**, set the threshold to greater than zero and save. Grafana then evaluates the LogQL query regularly.

12.3 The Attack Drill

Now you play through a break-in across all layers. Keep your dashboard open. You take on the role of an attacker who has gained a remote shell in the `frontend`.

Before you start, get out your notes from Chapter 1. Do your expectations hold up?

Phase 1: Looking Around

MAC Step 3

```
kubectl exec -it -n demo deploy/frontend -- sh
```

This simulates the break-in. **Falco** immediately reports `Terminal shell in container`.

POD Step 4

```
id
```

You are user 100, not root. That is **PSA**: `restricted` requires `runAsNonRoot`.

POD Step 5

```
ls /var/run/secrets/kubernetes.io/serviceaccount/
```

Error: the directory does not exist. That is **RBAC** from Chapter 3: `automountServiceAccountToken: false`. Without a token, the attacker cannot talk to the API server.

Phase 2: Spreading Out

POD Step 6

```
curl -s -m 3 -o /dev/null -w '%{http_code}\n' \
  http://kps-grafana.monitoring.svc.cluster.local/
```

Timeout (`000`). **Cilium** blocks it, because this path is not allowed.

POD Step 7

```
curl -s -m 3 -o /dev/null -w '%{http_code}\n' \
  http://backend:8080/admin
```

Expected: `403` . **Cilium** only lets `GET /` through.

Phase 3: Exfiltrating Data

POD Step 8

```
curl -s -m 5 -o /dev/null -w '%{http_code}\n' https://example.org/
```

Timeout (`000`). **Cilium** only allows `example.com` . The way out is closed.

Phase 4: Gaining a Foothold

POD Step 9

```
echo test > /usr/bin/hintertuer
```

`Permission denied` . You are not root, and the system directory does not belong to you.

POD Step 10

```
exit
```

The attacker is stuck.

Phase 5: The Detour Through the API Server

Imagine the attacker has stolen alice's credentials. Her certificate from Chapter 3 has expired by now, which already shows the value of short lifetimes. For the drill, you therefore act as alice with `--as` :

MAC Step 11

```
kubectl run ausbruch -n demo --image=busybox:1.36 \
  --privileged --as=alice --as-group=plattform
```

alice may create pods here, but **PSA** rejects the privileged pod.

MAC Step 12

```
kubectl get secrets -n kube-system --as=alice --as-group=plattform
```

`Forbidden` . **RBAC** only applies in `demo` . The attempt is now recorded in the **audit log**.

12.4 The Investigation

Switch roles: you are now the defense. Answer the questions using only Grafana.

Question	Where you look
When did the incident start, and in which pod?	Falco in Loki
Which connections were attempted?	Hubble
Did anyone access secrets?	Audit log in Loki
Who attempted a privileged pod?	Audit log, `responseStatus_code="403"`
What happened in the API request?	Tempo, if captured

MAC

Step 13

```
kubectl -n kube-system exec ds/cilium -c cilium-agent -- \
  hubble observe --from-pod demo/frontend \
  --verdict DROPPED --last 20
```

Shows all blocked attempts by the `frontend`. Hubble only keeps the most recent flows in memory; for a longer period, you use the metrics.

This query shows every denial for alice:

```
{job="audit"} | json
  | user_username="alice"
  | responseStatus_code="403"
```

WHAT YOU SHOULD SEE

You can tell the story of the attack: when it started, what was attempted and where each layer stepped in. That is exactly the difference between a collection of tools and an architecture.

12.5 What the Drill Showed

Action	Prevented by	Detected by
Shell in the container	nothing	Falco
Root privileges	PSA	Audit log
Stealing a token	RBAC, no mount	nothing needed
Reaching other services	Cilium	Hubble
Sending to the internet	Cilium	Hubble
Privileged pod	PSA	Audit log
Reading secrets	RBAC	Audit log

Note the first row: nothing prevented the shell itself. The break-in gains nothing because of how the other layers work together.

TRY IT YOURSELF

Look for the weaknesses in your own architecture. What if the attacker had landed in the `backend` instead of the `frontend`? What can someone do with your admin access on the Mac? Which of these gaps does none of the tools close? Write down three points for your next round of learning.

WHAT YOU LEARNED

- Individual tools only become an architecture when they work together.
- Not every layer prevents; some detect, and both roles are needed.
- A drill shows whether your assumptions hold.
- A dashboard without alerts only helps if someone looks at it.

CHAPTER 13

Operations and Troubleshooting

You keep the stack running, back up what matters and find errors with a fixed routine.

Time: 30 minutes of reading, then use it as a reference

13.1 The Routine After Every Restart

After the VM restarts, everything comes back up on its own.

MAC Step 1

```
kubectl get nodes
kubectl get pods -A | grep -vE 'Running|Completed'
```

The second command shows only pods that are not healthy. The list should be empty after a few minutes.

13.2 What You Need to Back Up

What	How	Where to store it
k3s server token	Password manager	Not in the cluster
etcd	Snapshot	Off the node
Configuration	This repo	Git

NODE Step 2

```
sudo k3s etcd-snapshot save --name manuell
```

Creates a snapshot of the datastore. This works because you chose `cluster-init` in Chapter 2.

NODE Step 3

```
sudo k3s etcd-snapshot ls
```

Shows all snapshots. k3s also creates one automatically every twelve hours.

PITFALL

An etcd snapshot alone is not enough for a restore. Because of `secrets-encryption` you also need the server token from `/var/lib/rancher/k3s/server/token`. Store it in your password manager. Also, a backup only counts as a backup once you have practiced the restore.

13.3 The Troubleshooting Routine

For every problem, work from the outside in. Don't guess, look.

1. **What isn't running?** `kubectl get pods -A`
2. **Why?** `kubectl describe pod <name> -n <ns>` ; the events are at the very bottom.
3. **What does the application say?** `kubectl logs <pod> -n <ns> --previous`
4. **Who blocked it?** Go through the table below.

Symptom	Likely layer	Check with
`Forbidden` when creating	RBAC	`kubectl auth can-i`
`violates PodSecurity`	PSA	Label on the namespace
`admission webhook denied`	Kyverno	`kubectl get polr -A`
Connection times out	Cilium	`hubble observe --verdict DROPPED`
HTTP 403 without an application log	Cilium L7	`hubble observe --verdict DROPPED`
Pod stuck in `ContainerCreating`	Secret or volume missing	`kubectl describe pod`
No logs in Grafana	Alloy or Loki	`kubectl logs ds/alloy -n monitoring`
No traces in Grafana	Alloy or Tempo	`kubectl logs statefulset/tempo -n monitoring`

13.4 The Emergency Brakes

When a security layer blocks operations, this is how you get out quickly. Use it only to fix the problem, and turn the layer back on afterwards.

Layer	Emergency brake
Kyverno	`kubectl delete vpol <name>` , in the worst case delete the webhook
Cilium policy	`kubectl delete cnp --all -n <ns>`
PSA	Set the label to `enforce=privileged`
Audit log	Comment out the lines in `config.yaml` , restart k3s

MAC — Step 4

```
kubectl label namespace demo --overwrite \
  pod-security.kubernetes.io/enforce=privileged
```

Example for PSA. `--overwrite` is required because the label already exists.

13.5 Updates

Stick to the order, because the components depend on each other.

1. **k3s:** only one minor version at a time. Check Cilium compatibility first.
2. **Cilium:** read the upgrade notes first.
3. **Kyverno:** `ClusterPolicy` goes away in 1.20. You already use `ValidatingPolicy`, so you are prepared.
4. **Everything else:** one at a time, with a `kubectl get pods -A` check in between.

If a later Cilium version supports Kubernetes 1.37, you can switch to the `stable` channel. To do that, you only change `INSTALL_K3S_CHANNEL` and run the install script again.

13.6 Keeping an Eye on Resources

MAC Step 5

```
kubectl top node
kubectl top pods -A --sort-by=memory | head -15
```

Shows the biggest memory consumers. Compare them with your plan from Chapter 1.

If memory gets tight, work through this list in order:

1. Lower the sampling of the API server traces or turn Tempo off.
2. Lower the Prometheus retention from two days to one.
3. Have Trivy scan only weekly: `scannerReportTTL: 168h`.
4. Add a second node.

WHAT YOU LEARNED

- A snapshot without the token and without a restore test is not a backup.
- Every layer has its typical error message, and its own emergency brake.

Where to Go From Here

You now have a complete foundation. These topics build on it well:

- **Image signatures:** Cosign and `ImageValidatingPolicy` in Kyverno.
- **GitOps:** Argo CD or Flux apply your manifests from the repo.
- **WireGuard:** Cilium encrypts the traffic between nodes.
- **Mutual TLS:** Cilium also authenticates individual pods to each other.
- **A second node:** this is how you learn high availability for etcd.
- **The optional part:** TLS and a secrets vault in the next three chapters.

Optional Part

This is where the main part ends. The following three chapters are optional; you do not need them for anything before this point.

They close two gaps that the main part leaves open: services talk to each other unencrypted, and secrets are created by scripts, with no history and no record of who read them.

Chapter	Contents
14	cert-manager: your own CA and automatic certificates
15	OpenBao: a vault with sealing, permissions and an audit log
16	External Secrets: vault values become Kubernetes secrets

The chapters build on each other and require Chapter 10. After every restart of the VM you have to unseal OpenBao by hand; Chapter 15 explains this.

CHAPTER 14

Optional: cert-manager

You run your own certificate authority in the cluster and have certificates issued and renewed automatically.

Time: 45 minutes **Requires:** Chapter 10

14.1 Why Automation

Certificates expire, usually at night and on weekends. cert-manager renews them on its own. You only describe which certificate you want, and it ends up as a Secret in the namespace.

Object	Meaning
`Issuer`, `ClusterIssuer`	Who issues? In the namespace or cluster-wide
`Certificate`	What do you want? Names, lifetime, target Secret
`Secret`	The result: `tls.crt`, `tls.key`, `ca.crt`

Your cluster sits in your home network, so Let's Encrypt cannot reach it. That is why you build your own private CA. The process is the same as with a public one.

14.2 Installing

MAC Step 1

```
helm upgrade --install cert-manager \
  oci://quay.io/jetstack/charts/cert-manager \
  --version v1.21.3 -n cert-manager --create-namespace \
  --set crds.enabled=true --wait
```

`crds.enabled=true` installs the required object types along with it. Without the flag they are missing, and `Certificate` would be unknown.

MAC Step 2

```
kubectl get pods -n cert-manager
```

Three pods should be running: controller, CA injector and webhook.

14.3 Building Your Own CA

The chain comes together in three steps: a self-signed issuer creates the root certificate, and that certificate then becomes the CA that signs everything else.

FILE security/tls/ca.yaml

```yaml
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: selfsigned
spec:
  selfSigned: {}
---
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: k3s2-root-ca
  namespace: cert-manager
spec:
  isCA: true
  commonName: k3s2 Root CA
  secretName: k3s2-root-ca
  duration: 87600h
  privateKey:
    algorithm: ECDSA
    size: 256
  issuerRef:
    name: selfsigned
    kind: ClusterIssuer
    group: cert-manager.io
---
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: k3s2-ca
spec:
  ca:
    secretName: k3s2-root-ca
```

`isCA: true` turns the certificate into a CA, and `duration: 87600h` makes it valid for ten years. For a `ClusterIssuer`, cert-manager looks for the Secret in the `cert-manager` namespace, which is why the root certificate lives there.

MAC — Step 3

```
kubectl apply -f security/tls/ca.yaml
```

Builds the CA.

MAC — Step 4

```
kubectl get clusterissuer
kubectl get certificate -n cert-manager
```

All entries should show `READY True`.

WHY IT MATTERS

The Secret `k3s2-root-ca` contains your CA's private key. Whoever has it can issue valid certificates for any name in your cluster. It is a good example of an object whose read access you must restrict tightly with RBAC.

14.4 The First Certificate

FILE `security/tls/demo-cert.yaml`

```yaml
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: backend-tls
  namespace: demo
spec:
  secretName: backend-tls
  duration: 2160h
  renewBefore: 720h
  dnsNames:
    - backend
    - backend.demo.svc
    - backend.demo.svc.cluster.local
  issuerRef:
    name: k3s2-ca
    kind: ClusterIssuer
    group: cert-manager.io
```

The certificate is valid for 90 days and is renewed 30 days before it expires. The `dnsNames` are all the names under which the Service is reachable. A mismatch leads to a certificate error on the client.

MAC Step 5

```
kubectl apply -f security/tls/demo-cert.yaml
kubectl get certificate -n demo
```

Should show `READY True` after a few seconds.

MAC Step 6

```
kubectl get secret backend-tls -n demo \
  -o jsonpath='{.data.tls\.crt}' | base64 -d \
  | openssl x509 -noout -subject -issuer -dates -ext subjectAltName
```

Reads the certificate from the Secret and shows issuer, validity period and names.

WHAT YOU SHOULD SEE

`issuer` reads `CN=k3s2 Root CA`, and `subjectAltName` lists your three DNS names.

14.5 Trying Out Renewal

MAC Step 7

```
kubectl get secret backend-tls -n demo \
  -o jsonpath='{.metadata.uid}{"\n"}'
```

Write down the UID.

MAC Step 8

```
kubectl delete secret backend-tls -n demo
```

Simulates a loss.

MAC Step 9

```
kubectl get secret backend-tls -n demo \
  -o jsonpath='{.metadata.uid}{"\n"}'
```

After a short while, the Secret is back, with a new UID. cert-manager constantly compares desired and actual state. That is exactly what the controller principle in Kubernetes means.

14.6 Making Grafana Reachable over HTTPS

So far you have used `port-forward`. Now you expose Grafana on the network through Traefik, with a certificate from your CA.

FILE `security/tls/grafana-ingress.yaml`

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: grafana
  namespace: monitoring
  annotations:
    cert-manager.io/cluster-issuer: k3s2-ca
spec:
  ingressClassName: traefik
  tls:
    - hosts:
        - grafana.k3s2.local
      secretName: grafana-tls
  rules:
    - host: grafana.k3s2.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: kps-grafana
                port:
                  number: 80
```

The annotation is enough: cert-manager sees the Ingress and creates the matching `Certificate` on its own.

MAC Step 10

```
kubectl apply -f security/tls/grafana-ingress.yaml
kubectl get certificate -n monitoring
```

The certificate `grafana-tls` is created automatically.

MAC Step 11

```
kubectl get secret k3s2-root-ca -n cert-manager \
  -o jsonpath='{.data.ca\.crt}' | base64 -d > k3s2-ca.crt
```

Fetches only the public part of the CA. You may share this file.

MAC Step 12

```
curl --cacert k3s2-ca.crt \
  --resolve grafana.k3s2.local:443:172.16.0.136 \
  https://grafana.k3s2.local/api/health
```

`--resolve` saves you the DNS entry, and `--cacert` trusts your CA.

WHAT YOU SHOULD SEE

The response is JSON with `"database": "ok"`. If you leave out `--cacert`, `curl` refuses the connection, because your Mac does not know the CA. That is exactly how it should be.

TRY IT YOURSELF

Add `172.16.0.136 grafana.k3s2.local` to `/etc/hosts` and open the address in your browser. Which warning appears, and what would you have to do in macOS Keychain Access to make it go away? Do not delete the file `k3s2-ca.crt`; you still need it.

WHAT YOU LEARNED

- cert-manager constantly reconciles certificates and renews them on its own.
- A private CA is built from a self-signed issuer and a CA issuer.
- The CA's key is one of the most valuable Secrets in the cluster.
- A single annotation on the Ingress is enough for automatic TLS.

CHAPTER 15

Optional: OpenBao as a Vault

You run your own secrets server with TLS and an audit log, and you understand how sealing, policies and login work.

Time: 75 minutes **Requires:** Chapter 14

15.1 Why a Vault

So far, your secrets exist as Kubernetes objects, created by scripts. That has limits: there is no history, no fine-grained permissions per path and no record of who read which value. OpenBao is an open source fork of Vault and solves exactly that.

Term	Meaning
Seal/Unseal	The vault starts sealed; the data is encrypted and inaccessible.
Unseal Keys	Key shares, a minimum number of which opens the vault.
Root Token	All-powerful initial token, only for the initial setup.
Secrets Engine	Storage location. `kv-v2` is a key-value store with versions.
Auth Method	How someone logs in, here via a Kubernetes ServiceAccount.
Policy	What a logged-in token may do, per path.

WHY IT MATTERS

Sealing protects the data if someone steals the disk or gets to the pod: without the unseal keys, the data is worthless. The price: after every restart, someone has to open the vault. You have to decide who does that. With a single node in your home network, that is you, by hand.

15.2 Preparing the TLS Certificate

MAC Step 1

```
kubectl create namespace openbao
```

The default `baseline` is enough; OpenBao does not need host access.

FILE `security/tls/openbao-cert.yaml`

```yaml
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: openbao-tls
  namespace: openbao
spec:
  secretName: openbao-tls
  duration: 2160h
  renewBefore: 720h
  dnsNames:
    - openbao
    - openbao.openbao
    - openbao.openbao.svc
    - openbao.openbao.svc.cluster.local
    - localhost
  ipAddresses:
    - 127.0.0.1
  issuerRef:
    name: k3s2-ca
    kind: ClusterIssuer
    group: cert-manager.io
```

`127.0.0.1` and `localhost` are needed because the `bao` command in the pod talks to the local address `https://127.0.0.1:8200`. The chart sets it up that way.

MAC

Step 2

```
kubectl apply -f security/tls/openbao-cert.yaml
kubectl get certificate -n openbao
```

The Secret `openbao-tls` must be ready before the pod starts.

15.3 Installing

FILE `openbao/values.yaml`

```yaml
global:
  tlsDisable: false

injector:
  enabled: false

server:
  standalone:
    enabled: true
    config: |
      ui = true

      listener "tcp" {
        address         = "[::]:8200"
        cluster_address = "[::]:8201"
        tls_cert_file   = "/openbao/tls/tls.crt"
        tls_key_file    = "/openbao/tls/tls.key"
      }

      storage "pebbledb" {
        path = "/openbao/data/pebbledb"
```

```
      }

      audit "file" "to-stdout" {
        description = "Audit log to standard output"
        options {
          file_path = "stdout"
        }
      }
  dataStorage:
    enabled: true
    size: 1Gi
    storageClass: local-path
  extraEnvironmentVars:
    BAO_CACERT: /openbao/tls/ca.crt
  volumes:
    - name: tls
      secret:
        secretName: openbao-tls
  volumeMounts:
    - name: tls
      mountPath: /openbao/tls
      readOnly: true
  resources:
    requests:
      cpu: 50m
      memory: 128Mi
    limits:
      memory: 256Mi

ui:
  enabled: true
```

`tlsDisable: false` turns encryption on. The injector is not needed, because External Secrets delivers the values. The audit device writes to standard output, and Alloy ships it to Loki. Values appear in the log only as hashes. The `log_raw` option from the docs would show them in plain text, so do not set it.

MAC Step 3

```
helm upgrade --install openbao openbao/openbao \
  -n openbao --version 0.30.7 -f openbao/values.yaml
```

The `openbao` repository is already registered on your machine; otherwise run `helm repo add openbao https://openbao.github.io/openbao-helm`.

PITFALL

Do **not** use `--wait` here. The pod only becomes `Ready` once the vault is unsealed. Helm would wait until the timeout and report an error, even though everything is running correctly.

MAC Step 4

```
kubectl get pods -n openbao
```

`openbao-0` is `Running`, but `0/1`. That is the correct state here.

15.4 Initializing and Unsealing

MAC Step 5

```
kubectl exec -it openbao-0 -n openbao -- bao operator init \
  -key-shares=3 -key-threshold=2
```

Generates the root key and splits it into three shares, two of which are enough. This command runs only once in the life of this vault.

PITFALL

The output shows the three unseal keys and the root token **only this one time**. Store them in your password manager right away, never in the repo and never in a file in Dropbox. If you lose the keys, all data in the vault is lost for good.

MAC Step 6

```
kubectl exec -it openbao-0 -n openbao -- bao operator unseal
```

Prompts for the first unseal key without echoing it. Do not type keys directly on the command line, or they end up in your shell history.

MAC Step 7

```
kubectl exec -it openbao-0 -n openbao -- bao operator unseal
```

Prompts for the second key. After the second of the two required shares, the vault is open.

MAC Step 8

```
kubectl exec -it openbao-0 -n openbao -- bao status
```

Shows the status.

WHAT YOU SHOULD SEE

`Initialized` is `true`, `Sealed` is `false`. `kubectl get pods -n openbao` now shows `1/1`.

15.5 First Steps in the Vault

MAC Step 9

```
kubectl exec -it openbao-0 -n openbao -- sh
```

Opens a shell in the pod. The following commands run inside it.

POD Step 10

```
bao login
```

Prompts for the root token without echoing it and keeps it in the pod for the current session.

POD Step 11

```
bao secrets enable -path=secret kv-v2
```

Enables a versioned key-value store at the path `secret/`. On a real server, this is not active by default.

POD Step 12

```
bao kv put secret/apps/demo \
  password="erstes-passwort" user=demo
```

Stores two values under one path.

POD Step 13

```
bao kv get secret/apps/demo
```

Reads them back.

POD Step 14

```
bao kv put secret/apps/demo \
  password="zweites-passwort" user=demo
```

Writes a new version.

POD Step 15

```
bao kv get -version=1 secret/apps/demo
```

Retrieves the old version. That is the history a Kubernetes Secret lacks.

15.6 Granting Permissions: the Policy

The root token may do anything; your applications should not. In OpenBao, everything is forbidden that has not been allowed, just like with RBAC.

POD Step 16

```
bao policy write eso-lesen - <<'EOF'
path "secret/data/apps/*" {
  capabilities = ["read"]
}
path "secret/metadata/apps/*" {
  capabilities = ["read", "list"]
}
EOF
```

Creates the policy `eso-lesen`. Important: with `kv-v2`, the values live under `secret/data/`, not under `secret/`. This is the most common source of errors.

The policy still needs someone to use it. Chapter 16 takes care of that.

POD Step 17

```
rm -f ~/.bao-token
exit
```

`bao login` stores the token in `~/.bao-token`. A root token does not belong in a file, so you delete it and leave the pod. At the end of every session with the root token, the rule is: delete the file.

15.7 Experiencing the Seal

MAC Step 18

```
kubectl delete pod openbao-0 -n openbao
```

Simulates a restart of the node.

MAC Step 19

```
kubectl get pods -n openbao
```

After a short while, the new pod is running, but it stays at `0/1`.

MAC Step 20

```
kubectl exec -it openbao-0 -n openbao -- bao status
```

`Sealed` is `true`. The data is still there, but inaccessible until you unseal. From now on, this procedure is part of every restart.

Now run `bao operator unseal` twice, as above.

MAC Step 21

```
kubectl logs -n openbao openbao-0 --tail=5
```

Here you see the audit log as JSON lines. In Grafana, you find it under `{namespace="openbao"}`.

UNSEAL AUTOMATICALLY?

In production, manual unsealing gets in the way. The solution is called auto-unseal: a cloud KMS or a second vault opens the first one. The key would then live somewhere else, though, and you need that infrastructure. For your lab, the manual approach is the more instructive choice.

TRY IT YOURSELF

Log in, write a second policy `nur-lesen-demo` that may read **only** `secret/data/apps/demo`, and create a token for it with `bao token create -policy=nur-lesen-demo -ttl=1h`. Test it with `BAO_TOKEN=... bao kv get secret/apps/demo`. What happens with `secret/apps/anderes`?

WHAT YOU LEARNED

- Sealing protects data at rest, but costs work at every restart.
- Unseal keys and the root token belong in the password manager, never in the repo.
- `kv-v2` stores versions and keeps values under `secret/data/`.
- The audit device shows who read what, without revealing the values.

CHAPTER 16

Optional: External Secrets

Values from OpenBao appear automatically as Kubernetes Secrets, and changes in the vault reach the cluster on their own.

Time: 60 minutes **Requires:** Chapter 15

16.1 How the Pieces Fit Together

Your applications only know regular Kubernetes Secrets. They should not need to know anything about OpenBao. The External Secrets Operator (ESO) builds the bridge:

1. In an `ExternalSecret`, you describe which values you need.
2. ESO logs in to OpenBao with a short-lived ServiceAccount token.
3. OpenBao verifies the token with the Kubernetes API server.
4. ESO fetches the values and writes a regular Secret.
5. If the value changes in the vault, ESO updates the Secret.

WHY IT MATTERS

Git now only holds the description of where a value lives, never the value itself. This means you can safely check in the entire configuration. The Secret in the cluster remains as a copy, but thanks to `secrets-encryption` from Chapter 2, it is encrypted in the datastore.

16.2 Installing

MAC Step 1

```
helm repo add external-secrets https://charts.external-secrets.io
helm repo update
```

Registers the repository.

MAC Step 2

```
helm upgrade --install external-secrets \
  external-secrets/external-secrets \
  -n external-secrets --create-namespace \
  --version 2.14.0 --wait
```

Installs the operator with the webhook and cert controller.

MAC Step 3

```
kubectl get pods -n external-secrets
```

Three pods should be running.

16.3 The Account for Access

ESO gets its own account. OpenBao will trust exactly this account.

MAC Step 4

```
kubectl create serviceaccount eso-openbao -n external-secrets
```

This account has no permissions in Kubernetes and does not need any. It only serves as an ID card toward OpenBao.

16.4 OpenBao Trusts Kubernetes

MAC Step 5

```
kubectl exec -it openbao-0 -n openbao -- sh
```

Shell in the pod. The vault must be unsealed.

POD Step 6

```
bao login
```

Log in with the root token.

POD Step 7

```
bao auth enable kubernetes
```

Enables login via ServiceAccount.

POD Step 8

```
bao write auth/kubernetes/config \
  kubernetes_host=https://kubernetes.default.svc:443
```

Tells OpenBao where the API server is. OpenBao takes the CA certificate and the reviewer token automatically from its own pod. For this, the chart has given its account the role `system:auth-delegator`.

POD Step 9

```
bao write auth/kubernetes/role/eso \
  bound_service_account_names=eso-openbao \
  bound_service_account_namespaces=external-secrets \
  audience=openbao \
  policies=eso-lesen \
  ttl=1h
```

Connects everything: the account `eso-openbao` from `external-secrets` gets the policy `eso-lesen`. `audience` defines who the token must be issued for. That way, a token for a different service cannot be misused here.

POD Step 10

```
rm -f ~/.bao-token
exit
```

Removes the stored root token and takes you back to the Mac.

16.5 Describing the Connection

FILE `security/secrets/store.yaml`

```
apiVersion: external-secrets.io/v1
kind: ClusterSecretStore
metadata:
  name: openbao
spec:
  provider:
    vault:
      server: https://openbao.openbao.svc.cluster.local:8200
      path: secret
      version: v2
      caProvider:
        type: Secret
        name: openbao-tls
        namespace: openbao
        key: ca.crt
      auth:
        kubernetes:
          mountPath: kubernetes
          role: eso
          serviceAccountRef:
            name: eso-openbao
            namespace: external-secrets
            audiences:
              - openbao
```

According to the ESO docs, OpenBao runs through the `vault` provider, because OpenBao speaks the same API. The CRDs now also have a separate `openBao` key, but this book sticks with the documented route. `caProvider` takes the trust from the certificate in Chapter 15. `audiences` must match the role in OpenBao.

MAC Step 11

```
kubectl apply -f security/secrets/store.yaml
kubectl get clustersecretstore openbao
```

The `STATUS` column shows `Valid`, and `READY` shows `True`.

PITFALL

If you see `InvalidProviderConfig`, `kubectl describe clustersecretstore openbao` helps. Common causes: the vault is sealed, the `audience` does not match, or the role names the wrong account.

16.6 The First External Secret

FILE `security/secrets/demo-db.yaml`

```yaml
apiVersion: external-secrets.io/v1
kind: ExternalSecret
metadata:
  name: demo-db
  namespace: demo
spec:
  refreshInterval: 1m
  secretStoreRef:
    name: openbao
    kind: ClusterSecretStore
  target:
    name: demo-db
  data:
    - secretKey: password
      remoteRef:
        key: apps/demo
        property: password
```

`refreshInterval: 1m` is short for learning; in production, an hour or more is enough. `remoteRef.key` is the path in the vault, without `secret/` and without `data/`. `property` selects the value within it. `target.name` is the name of the resulting Secret.

MAC Step 12

```
kubectl apply -f security/secrets/demo-db.yaml
kubectl get externalsecret -n demo
```

`STATUS` should show `SecretSynced`.

MAC Step 13

```
kubectl get secret demo-db -n demo \
  -o jsonpath='{.data.password}' | base64 -d; echo
```

Shows the value from the vault. You never created a Secret file.

16.7 Changing and Following Along

MAC Step 14

```
kubectl exec -it openbao-0 -n openbao -- sh
```

Back into the vault.

POD Step 15

```
bao login
```

Log in.

POD Step 16

```
bao kv put secret/apps/demo \
  password="drittes-passwort" user=demo
```

Changes the value. From now on, this is the only place where it is maintained.

POD Step 17

```
rm -f ~/.bao-token
exit
```

Removes the token and takes you back to the Mac.

MAC Step 18

```
kubectl get secret demo-db -n demo \
  -o jsonpath='{.data.password}' | base64 -d; echo
```

Within one minute at most, the new value appears.

PITFALL

The Secret changes, but the pods do not necessarily. A Secret mounted as a **volume** is updated in the container after a short while, but the application has to reread the file. A Secret exposed as an **environment variable**, on the other hand, never changes in the running process. Then only `kubectl rollout restart` helps.

16.8 Who May Do What Now?

Check the model once against what you have learned:

MAC Step 19

```
kubectl auth can-i get secrets -n demo \
  --as=system:serviceaccount:demo:app
```

Expected: `no`. The account `app` may only read pods. That is exactly why you scoped it so tightly in Chapter 3.

Also try the audit view in Grafana:

```
{namespace="openbao"} |= "secret/data/apps/demo"
```

This shows you every read access by ESO to this path, with timestamp and account.

16.9 Moving Loki's Credentials

Now you take over a real application. Loki's S3 Secret from Chapter 10 moves into the vault.

MAC Step 20

```
ACCESS=$(kubectl get secret loki-s3 -n monitoring \
  -o jsonpath='{.data.S3_ACCESS_KEY}' | base64 -d)
SECRET=$(kubectl get secret loki-s3 -n monitoring \
  -o jsonpath='{.data.S3_SECRET_KEY}' | base64 -d)
```

Reads the values into shell variables.

MAC Step 21

```
kubectl exec -i openbao-0 -n openbao -- sh -c 'bao login -no-print -'
```

Reads the token from standard input. Paste it and press `Ctrl-D`. That way it does not end up in your history.

MAC Step 22

```
printf '{"S3_ACCESS_KEY":"%s","S3_SECRET_KEY":"%s"}' "$ACCESS" "$SECRET" \
  | kubectl exec -i openbao-0 -n openbao -- \
    bao kv put secret/apps/loki-s3 -
```

Transfers the values as JSON via standard input. They show up neither in the process list nor in the log.

MAC Step 23

```
unset ACCESS SECRET
kubectl exec openbao-0 -n openbao -- rm -f /home/openbao/.bao-token
```

Removes the values from the shell and the token from the pod. Check with `kubectl exec openbao-0 -n openbao -- sh -c 'echo $HOME'` whether the home directory has that name on your system.

FILE `security/secrets/loki-s3.yaml`

```
apiVersion: external-secrets.io/v1
kind: ExternalSecret
metadata:
  name: loki-s3
  namespace: monitoring
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: openbao
    kind: ClusterSecretStore
  target:
    name: loki-s3
    creationPolicy: Merge
  dataFrom:
    - extract:
        key: apps/loki-s3
```

`dataFrom.extract` takes over all keys of the path. `creationPolicy: Merge` writes into the existing Secret, because that Secret still belongs to you and not to ESO.

MAC

Step 24

```
kubectl apply -f security/secrets/loki-s3.yaml
kubectl get externalsecret -n monitoring
```

From now on, OpenBao is the source of truth for this Secret.

TRY IT YOURSELF

Following the same pattern, migrate the Grafana password. Think about this first: what would have to happen for Grafana to actually pick up a changed password?

WHAT YOU LEARNED

- ESO translates vault paths into regular Kubernetes Secrets.
- Git only holds the reference to a value, never the value itself.
- OpenBao verifies ServiceAccount tokens with the API server, and `audience` binds them to one service.
- Environment variables never change in the running process.
- Kubernetes Secrets are the copy; the vault remains the source.

Appendix

Versions

As of November 6, 2026. Update deliberately and one at a time, not everything at once.

Component	Helm chart	App version
k3s (channel `v1.36`)	–	v1.36.6+k3s1
Cilium	1.20.4	1.20.4
kube-prometheus-stack	92.18.0	from your README
Loki (`grafana-community/loki`)	21.0.0	3.7.9
Tempo (`grafana-community/tempo`)	3.2.1	3.1.1
Alloy (`grafana/alloy`)	1.15.0	v1.21.3
Kyverno	3.9.2	v1.19.2
Falco	9.2.2	0.45.2
Trivy Operator	0.38.1	0.36.1
cert-manager (optional)	v1.21.3	v1.21.3
OpenBao (optional)	0.30.7	v2.7.2
External Secrets Operator (optional)	2.14.0	v2.14.0

Your Repo at the End

```
k3s/
├── k3s/config.yaml              Node configuration
├── cilium/
│   ├── values.yaml
│   └── metrics-values.yaml
├── monitoring/
│   ├── kps-k3s-values.yaml
│   ├── loki-values.yaml
│   ├── tempo-values.yaml
│   ├── alloy-values.yaml
│   └── config.alloy
├── seaweedfs/                   Existing
├── kyverno/values.yaml
├── falco/values.yaml
├── trivy/values.yaml
├── openbao/values.yaml
└── security/
    ├── audit-policy.yaml
```

```
├── psa.yaml
├── alerts.yaml
├── tracing.yaml
├── rbac/       ServiceAccounts
├── kyverno/    Policies and tests
├── netpol/     Cilium policies
├── demo/       Lab application
├── tls/        Optional: CA, certificates
└── secrets/    Optional: ExternalSecrets
```

No unseal keys, tokens or private keys may be stored in this repo. Check this before every commit.

Cheat Sheet

Question	Command
May X do this?	`kubectl auth can-i <verb> <res> --as=<X>`
What may X do?	`kubectl auth can-i --list --as=<X>`
Who would a PSA change affect?	`kubectl label --dry-run=server ...`
Which policies exist?	`kubectl get vpol`
Which violations exist?	`kubectl get polr -A`
What was blocked?	`hubble observe --verdict DROPPED`
Which CVEs exist?	`kubectl get vuln -A -o wide`

For `hubble` and `cilium-dbg`, always use the prefix:

```
kubectl -n kube-system exec ds/cilium -c cilium-agent --
```

LogQL Cheat Sheet

Goal	Query
Logs of a namespace	`{namespace="demo"}`
Contains text	`{namespace="demo"} |= "error"`
Excludes text	`{namespace="demo"} != "health"`
Parse JSON	`{job="audit"} | json`
Filter a field	`... | json | verb="delete"`
Count	`count_over_time({...}[5m])`

Official Documentation

- Cilium: https://docs.cilium.io/en/stable/

- k3s hardening: https://docs.k3s.io/security/hardening-guide
- Kubernetes Pod Security: https://kubernetes.io/docs/concepts/security/pod-security-standards/
- Kubernetes audit: https://kubernetes.io/docs/tasks/debug-application-cluster/audit/
- Kubernetes traces: https://kubernetes.io/docs/concepts/cluster-administration/system-traces/
- Kubernetes RBAC: https://kubernetes.io/docs/reference/access-authn-authz/rbac/
- Kyverno: https://kyverno.io/docs/
- Falco: https://falco.org/docs/
- Trivy Operator: https://aquasecurity.github.io/trivy-operator/
- cert-manager: https://cert-manager.io/docs/
- OpenBao: https://openbao.org/docs/
- External Secrets: https://external-secrets.io/
- Loki: https://grafana.com/docs/loki/latest/
- Tempo: https://grafana.com/docs/tempo/latest/
- Alloy: https://grafana.com/docs/alloy/latest/

SECURING KUBERNETES

An empty Ubuntu server, a laptop and one goal: a Kubernetes cluster you can trust.

This book walks you through building an end-to-end security architecture on k3s, layer by layer. No long script to copy, but every command on its own, with an explanation of why it is needed and how you can tell that it worked.

INSIDE

Foundation	k3s with Cilium instead of Flannel and kube-proxy
Access	RBAC and Pod Security Admission
Policy	Kyverno and Cilium default deny
Runtime	Falco detects suspicious behavior
Vulnerabilities	Trivy Operator scans continuously
Evidence	Audit logs, Grafana, Loki and Tempo
Practice	An attack drill against your own cluster

Thomas Zachmann
Author on Leanpub



Hands-on guide · As of November 6, 2026